

MicroPython チュートリアル

内容目次

内容目次	1
2. プログラミング学習	2
4.1 始める前に：重要な注意事項	2
4.2 Thonny IDE のインストール	2
4.3 CH340 USB ドライバのインストール	5
4.4 Thonny IDE を使った MicroPython ファームウェアの書き込み	8
4.5 Thonny IDE の概要	12
4.6 初めてのコードを実行する - Thonny で現在のコードを実行	14
4.7 ESP32 ボードへのコードアップロード - ESP32 でコードを実行	16
方法 1: main.py として直接保存/アップロード	16
方法 2: main.py が他のファイルを読み出す	18
4.8 コード例	24
4.8.1 曲を再生する	24
4.8.2 サーボを動かす	25
4.8.3 ジョイスティックの値を読み取る	27
4.8.4 ジョイスティックで eArm を制御する(記録機能付き)	28
4.8.5 Web アプリの値を読み取る	30
4.8.6 Web アプリで eArm ロボットを制御する	32
3. ファームウェアの書き込み	36
5.1 書き込みツール	37
5.2 eArm ファームウェア（工場出荷時ファームウェア）	37
5.2.1 eArm ファームウェアの書き込み	38
5.3 ジョイスティック制御ファームウェア（記録機能付き）	39
5.3.1 ジョイスティック制御ファームウェアの書き込み	39
4. よくある問題と解決策	42
6.1 MacOS への Thonny IDE のインストール方法	42
6.2 ロボットアームが動作しない	45
6.3 ポート接続・デバイス認識エラー	45
6.4 MicroPython ファームウェア・インタープリター設定エラー	47
6.5 ランタイムエラー（ESP32 でのコード実行）	48
6.6 ファイルシステム・デバイス表示の異常	52
6.7 シリアルポートの占有・接続切断の問題	53
5. お問い合わせ	54

プログラミング学習

1.1 始める前に：重要な注意事項

ESP32-C3 コントローラーボードには、あらかじめロボットアームを操作するためのファームウェアが書き込まれています。このファームウェアにより、「組立・取扱説明書」に記載されているロボットアームの機能をご利用いただけます。書き込まれたファームウェアは、Thonny IDE を使用して MicroPython ファームウェアを書き込むまで保持されます。

****ご注意：****

MicroPython ファームウェアの書き込みはいつでも元に戻すことができます。ESP32-C3 ボードに MicroPython を書き込んだ後でも、Arduino IDE を使用して Arduino スケッチを書き込むだけで、いつでも Arduino 環境に戻すことができます。

チュートリアルパッケージには、MicroPython のサンプルプログラムが含まれています。先にチュートリアルパッケージをダウンロードし、保存先のフォルダーを覚えておいてください。後の手順でサンプルプログラムを開く際に使用します。

01_MicroPython > Example_Codes			
 buzzer	2026/1/6 17:47	Python file	4 KB
 hello_world	2026/1/7 11:55	Python file	1 KB
 joystick	2026/1/8 21:24	Python file	4 KB
 joystick_control_eArm	2026/3/26 22:43	Python file	15 KB
 main	2026/1/7 16:32	Python file	4 KB
 servo	2026/1/7 15:14	Python file	6 KB
 song	2026/1/6 17:46	Python file	7 KB
 web_app	2026/1/8 16:33	Python file	11 KB
 web_app_control_eArm	2026/1/9 8:56	Python file	16 KB

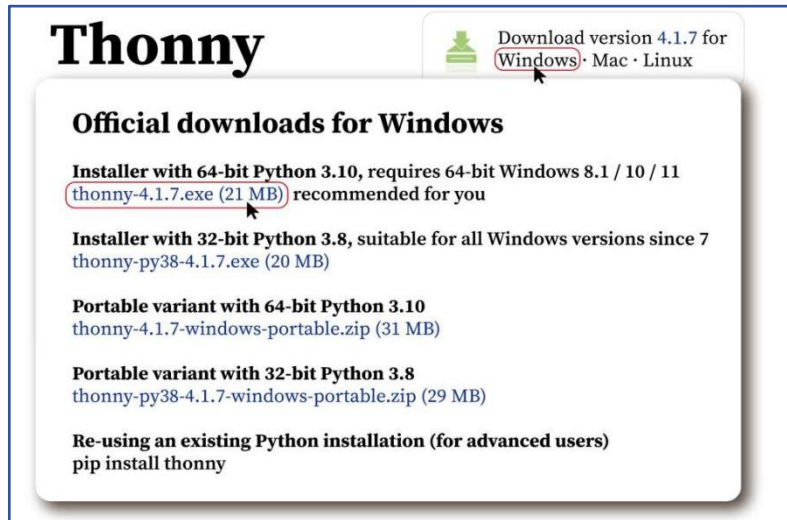
1.2 Thonny IDE のインストール

Thonny 公式ウェブサイト: <https://thonny.org>

Thonny は Windows、macOS、Linux の 3 つの主要なオペレーティングシステムをサポートしています。お使いの OS に該当するセクションに従ってください。

オペレーティングシステム	インストール方法
Windows	I) Thonny IDE のインストール - Windows PC
Mac OS	II) Thonny IDE のインストール - Mac OS

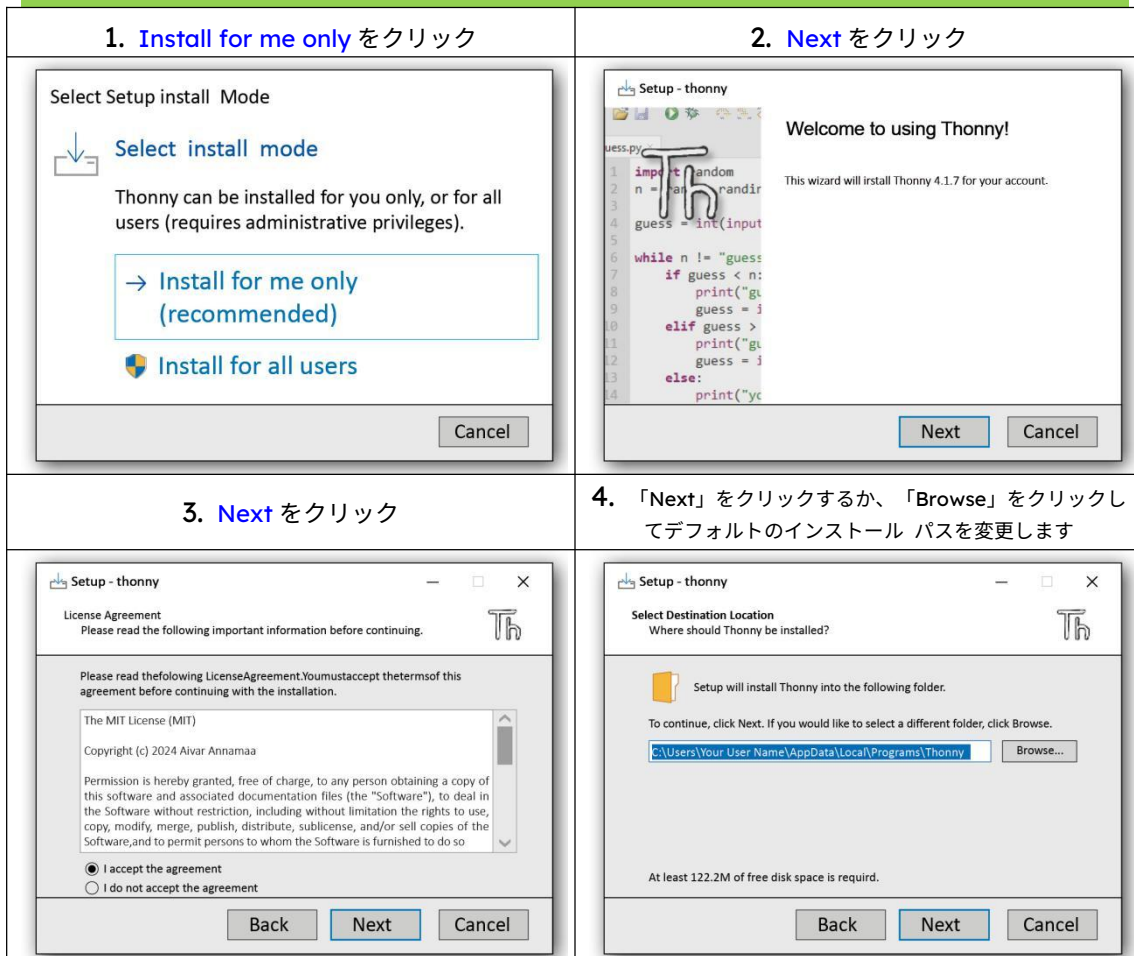
I) Thonny IDE のインストール - Windows PC

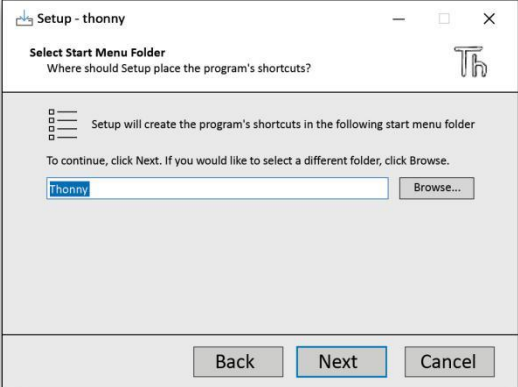
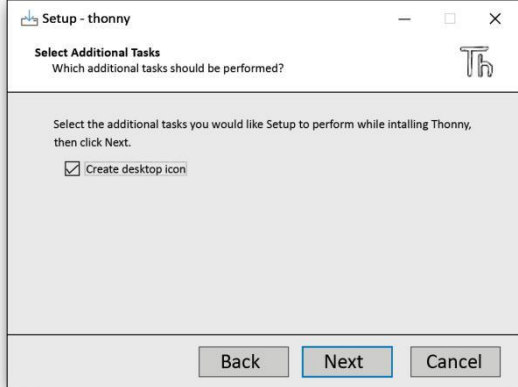
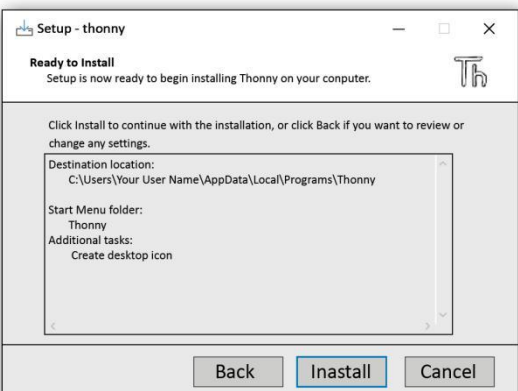
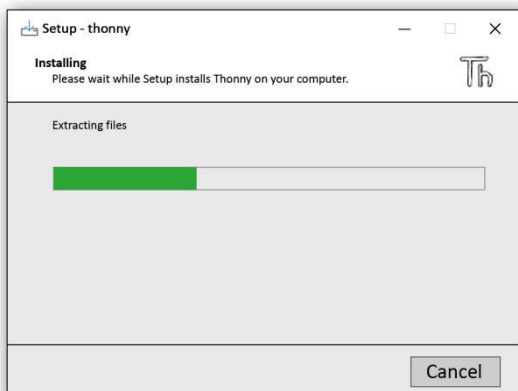
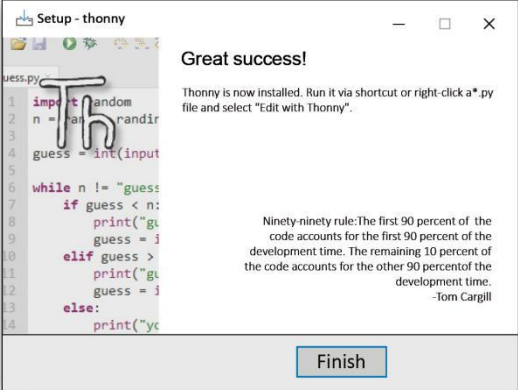


1. インストーラー thionny-4.1.7.exe をダウンロードし、ダブルクリックして実行します



2. 「Next」をクリックし続けてインストールを完了します。



5. Next をクリック	6. 「Create desktop icon」を選択し、「Next」をクリックします
	
7. Install をクリック	8. インストールが完了するまで数秒お待ちください
	
9. 仕上げる	
	

3. インストール後、デスクトップのアプリケーションアイコンを見つけて実行します

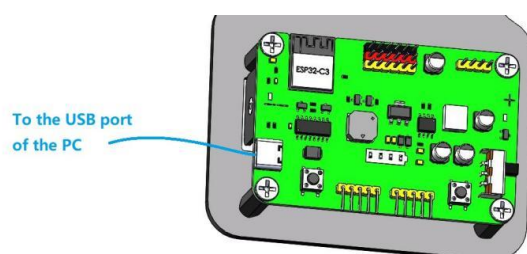


1.3 CH340 USB ドライバのインストール

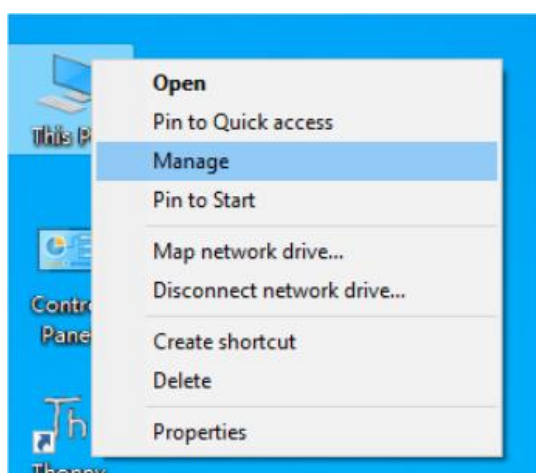
オペレーティングシステム	インストール方法
Windows	I) Windows システムへのドライバインストール
Linux	II) Linux システムへのドライバインストール
Mac OS	III) MAC システムへのドライバインストール

I) Windows システムへのドライバインストール

1. USB ケーブルでコンピューターと ESP32 を接続します。

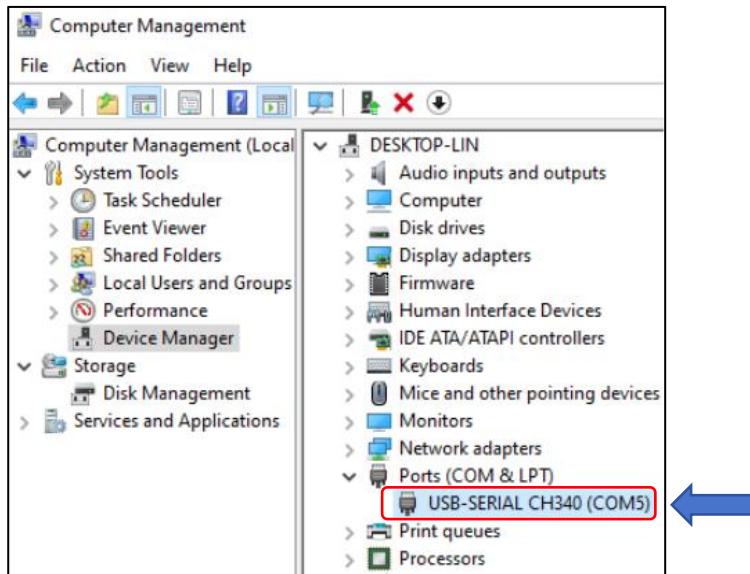


2. コンピューターのメインインターフェースで「PC」を選択し、右クリックして「管理」を選択します



3. CH340 がインストールされているか確認します

ドライバが正しくインストールされている場合、ESP32 ボードが USB ケーブルでコンピューターに接続されると、「デバイスマネージャー」→「ポート (COM と LPT) 」のパスに「USB-Serial CH340 (COMX) 」というデバイスが表示されます。下図を参照してください。

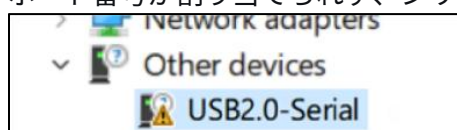


注：ドライバの表示記号は「USB-SERIAL CH340 (COMx)」です。「x」は任意の数字で、コンピューターが ESP32 デバイスに割り当てた番号によって異なります。

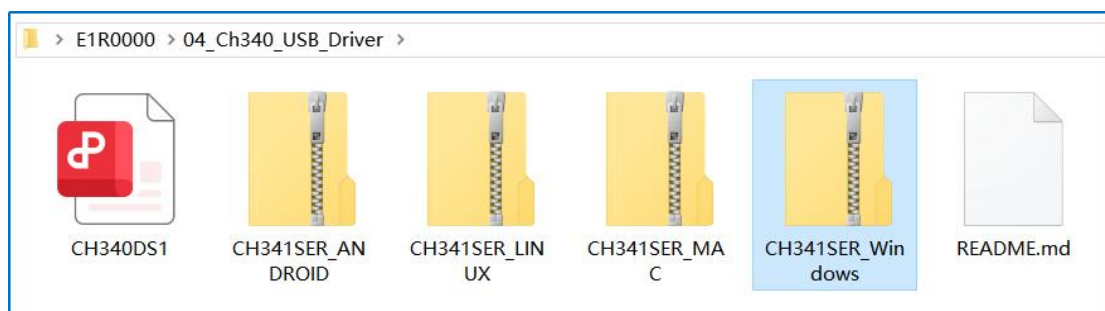
ドライバが既にインストールされている場合は、ドライバのインストール手順をスキップして次の章に進んでください。

4.CH340 ドライバを手動でインストールする

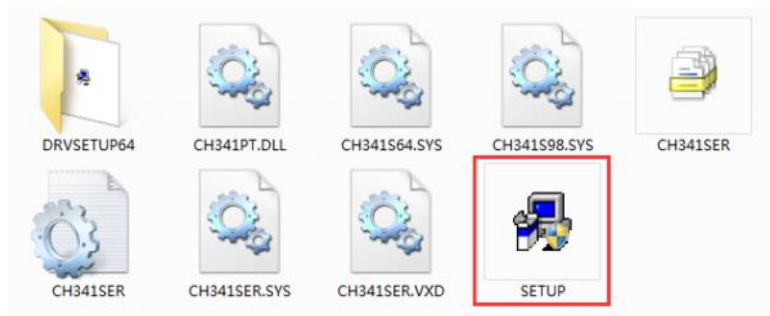
CH340 ドライバがインストールされていない場合、Windows のデバイスマネージャーには通常、「ポート (COM と LPT) 」または「その他のデバイス」の下に「USB2.0-Serial」または「不明なデバイス」が黄色い感嘆符付きで表示されます。これは、コンピューターがハードウェアを検出したが正しく使用できないことを示しています。その結果、COM ポート番号が割り当てられず、シリアル通信ができなくなります。



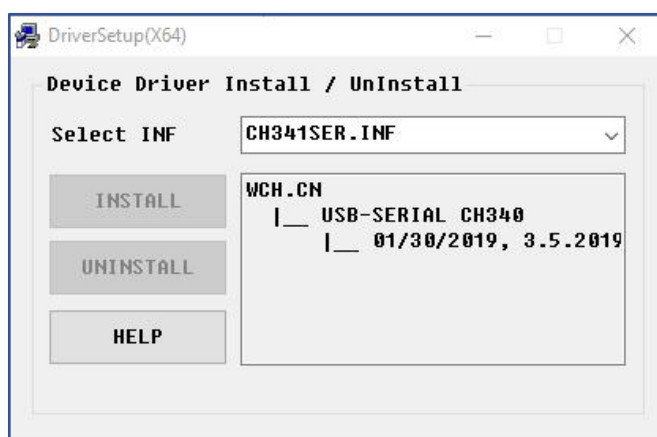
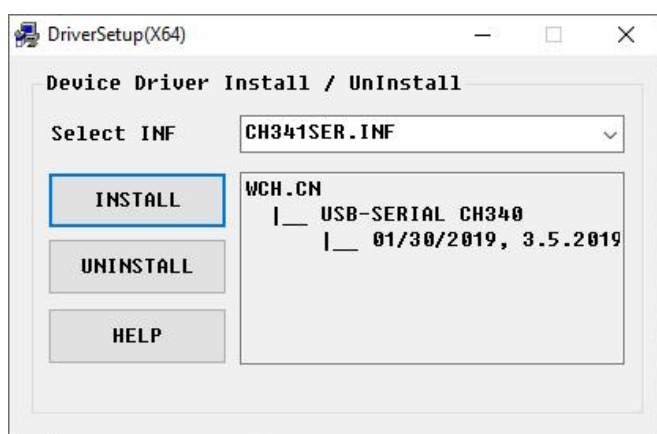
ドライバファイルはチュートリアルパッケージに含まれています：



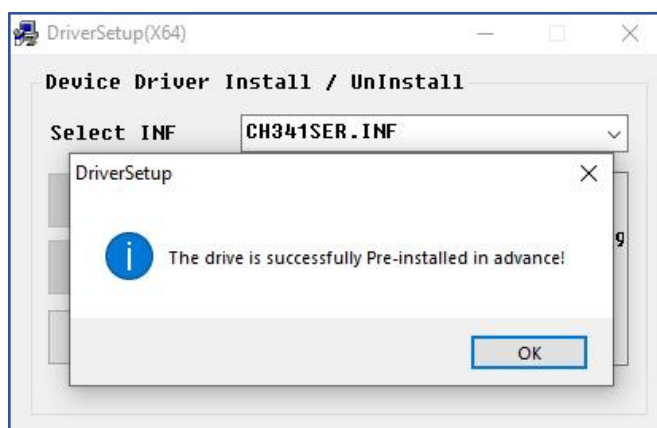
提供されたファイルを解凍し、SETUP をダブルクリックしてインストーラーを実行します：



「INSTALL」をクリックし、インストールが完了するまで待ちます。



インストール成功。すべてのインターフェースを閉じます。



1.4 Thonny IDE を使った MicroPython ファームウェアの書き込み

Thonny IDE から ESP32 にコードをアップロードするには、まず ESP32 に MicroPython ファームウェアが書き込まれ、Thonny が MicroPython インタープリターを使用するように設定されていることを確認する必要があります。

このチュートリアルで使用するファームウェア
は **ESP32_GENERIC_C3-20251209-v1.27.0.bin** です。

このマイクロパイソンファームウェアファイルは、チュートリアルパッケージに含まれています。



または、公式ウェブサイトからダウンロードすることもできます：

ESP32-C3 用 microPython ファームウェア一覧：

https://micropython.org/download/ESP32_GENERIC_C3/

Firmware

Releases

v1.27.0 (2025-12-09) .bin / [.app-bin] / [.elf and .map] / [Release notes] (latest)

v1.26.1 (2025-09-11) .bin / [.app-bin] / [.elf and .map] / [Release notes]

v1.26.0 (2025-08-09) .bin / [.app-bin] / [.elf and .map] / [Release notes]

v1.25.0 (2025-04-15) .bin / [.app-bin] / [.elf and .map] / [Release notes]

v1.24.1 (2024-11-29) .bin / [.app-bin] / [.elf and .map] / [Release notes]

v1.24.0 (2024-10-25) .bin / [.app-bin] / [.elf and .map] / [Release notes]

v1.23.0 (2024-06-02) .bin / [.app-bin] / [.elf and .map] / [Release notes]

v1.22.2 (2024-02-22) .bin / [.app-bin] / [.elf and .map] / [Release notes]

v1.22.1 (2024-01-05) .bin / [.app-bin] / [.elf and .map] / [Release notes]

v1.22.0 (2023-12-27) .bin / [.app-bin] / [.elf and .map] / [Release notes]

v1.21.0 (2023-10-05) .bin / [.app-bin] / [.elf and .map] / [Release notes]

v1.20.0 (2023-04-26) .bin / [.elf and .map] / [Release notes]

v1.19.1 (2022-06-18) .bin / [.elf and .map] / [Release notes]

v1.18 (2022-01-17) .bin / [.elf and .map] / [Release notes]

v1.17 (2021-09-02) .bin / [.elf and .map] / [Release notes]

Preview builds

These are automatic builds of the development branch for the next release.

v1.28.0-preview.121.g2b4b05a422 (2026-02-03) .bin / [.app-bin] / [.elf] / [.map]

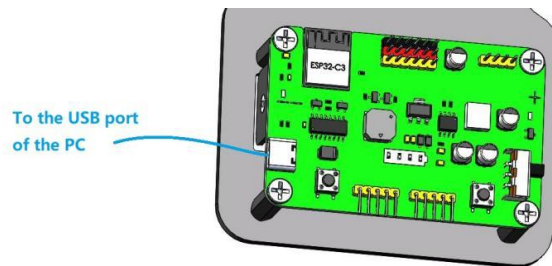
v1.28.0-preview.117.g023a49c55e (2026-01-31) .bin / [.app-bin] / [.elf] / [.map]

v1.28.0-preview.110.gaf76da96a3 (2026-01-29) .bin / [.app-bin] / [.elf] / [.map]

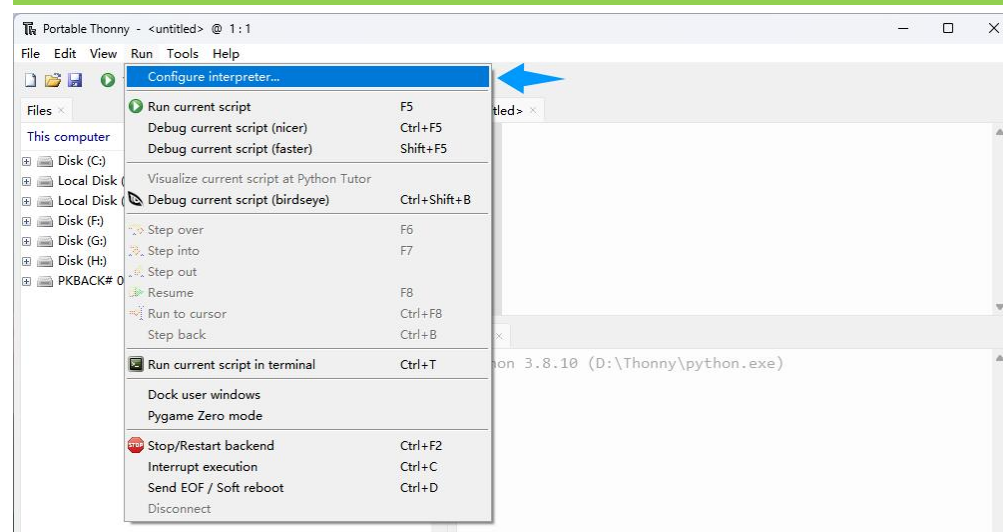
v1.28.0-preview.109.gbe15be3ab5 (2026-01-27) .bin / [.app-bin] / [.elf] / [.map]

次の手順に従ってください:

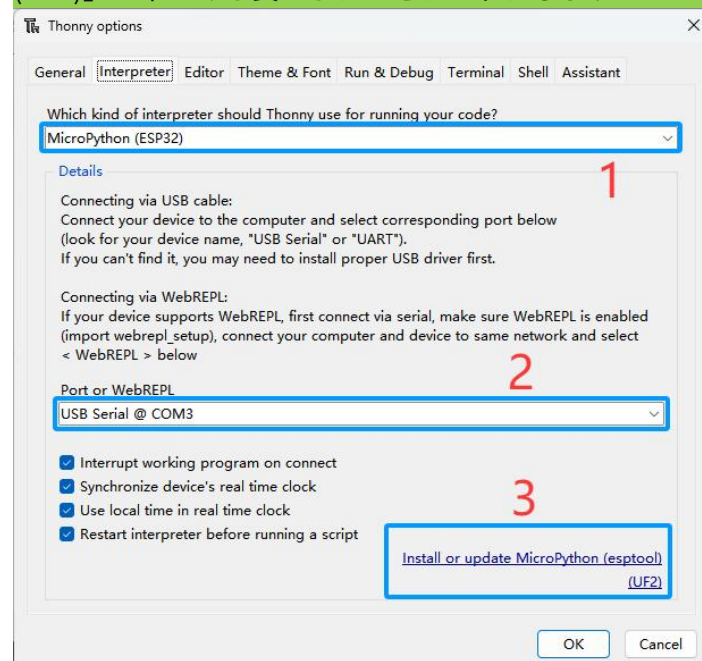
1) ESP32 ボードをコンピューターに接続します



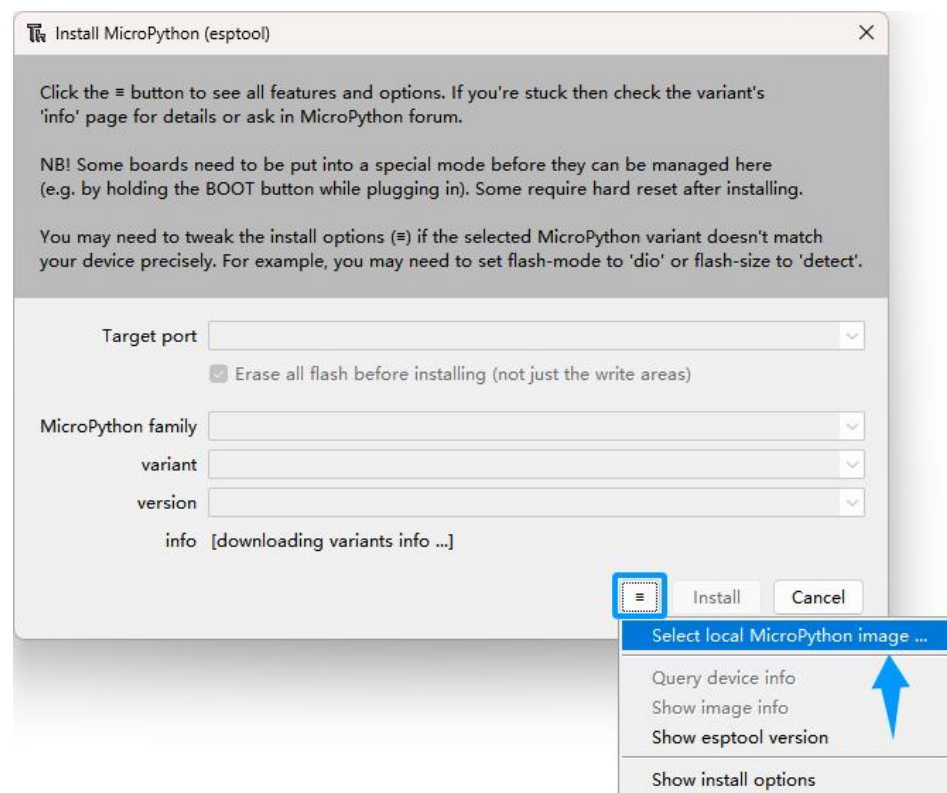
2) Thonny IDE を開きます。「Run」をクリックし、「Configure interpreter...」を選択します



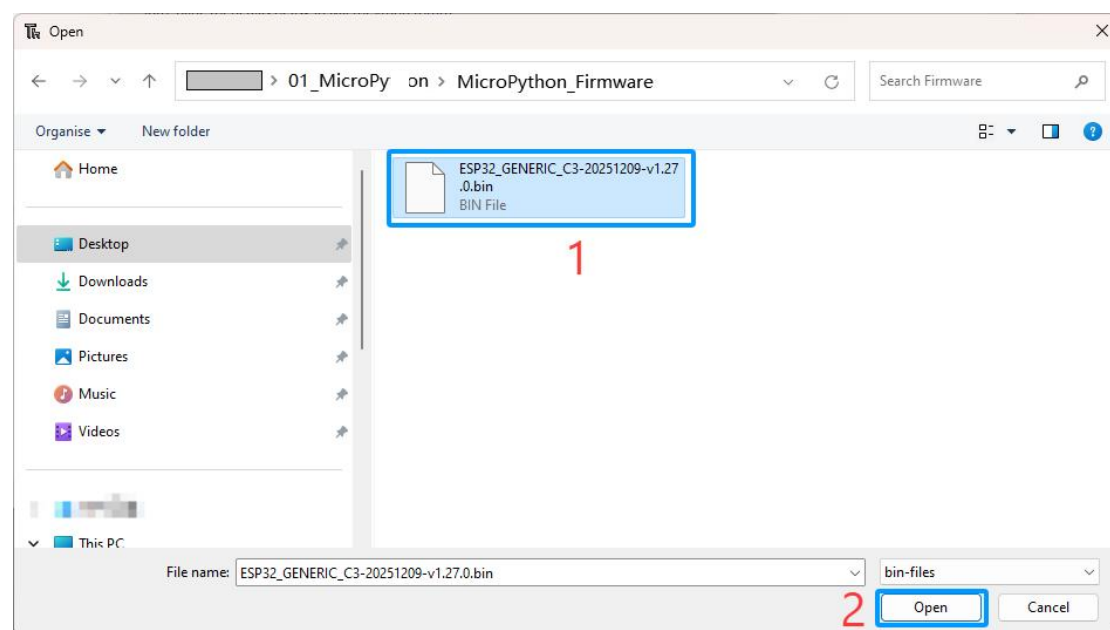
3) 「Micropython (ESP32)」を選択し、「USB Serial @ COMX (デバイスマネージャーで割り当てられた番号に依存)」を選択し、「Install or update MicroPython (esptool) (UF2)」の下にある長いボタンをクリックします



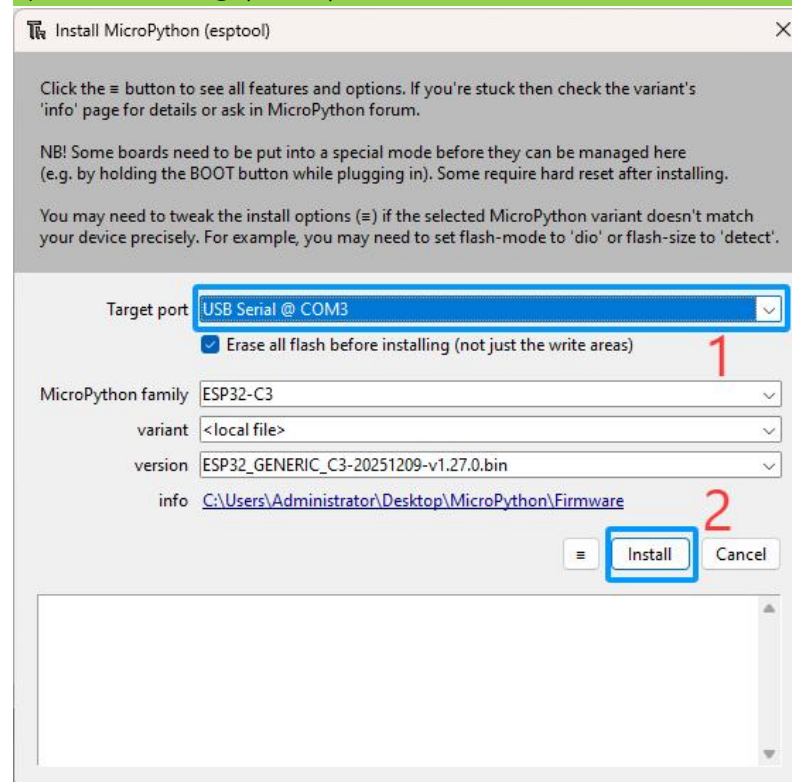
4) 以下のダイアログボックスが表示されます。「Select local MicroPython image...」を選択します



5) チュートリアルパッケージで提供されている microPython ファームウェア「ESP32_GENERIC_C3-20251209-v1.27.0.bin」を選択します。「Open」をクリックします



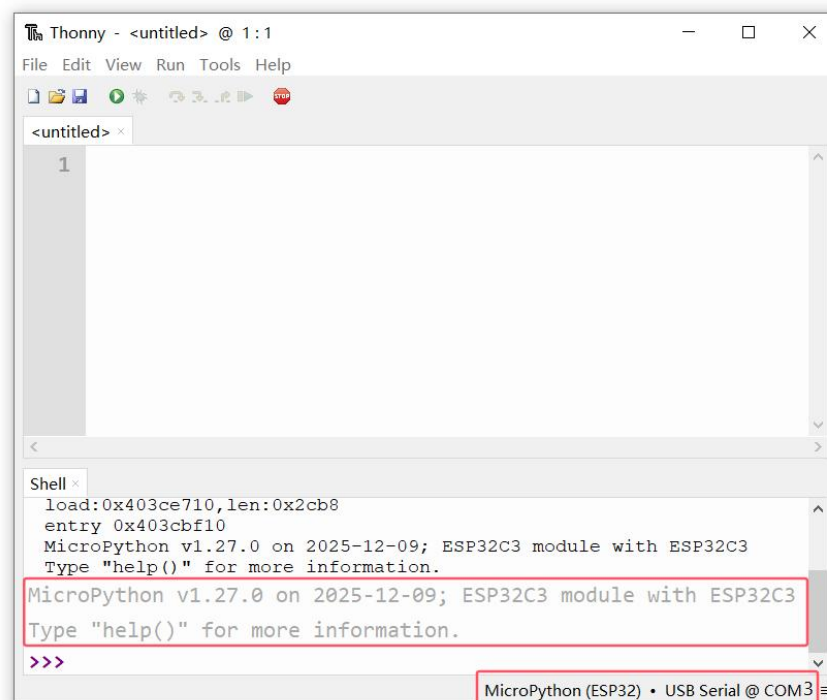
6) 「USB Serial @ (COM3)」を選択し、「Install」をクリックします



Wait for the installation to be done, and then click "Close".

7) インストールのテスト

インストールが完了すると、ウィンドウを閉じることができます。シェルに以下のメッセージが表示され（下図参照）、右下隅に使用しているインタープリターと COM ポートが表示されるはずです。



REPL (シェル)

>>>記号も表示されるはずです

このシンボルは MicroPython REPL (Read-Eval-Print-Loop) プロンプトを表します。このシンボルは、MicroPython (MicroPython ファームウェアがインストールされたボード) とリアルタイムで対話モードでやり取りしていることを意味します。つまり、コマンドを直接入力して実行できることを意味します。

プロンプト>>>の後にシェルに **print('Hello')** と入力し、Enter キーを押します。

コマンドを入力するとすぐに、シェルに **Hello** と表示されるはずです。

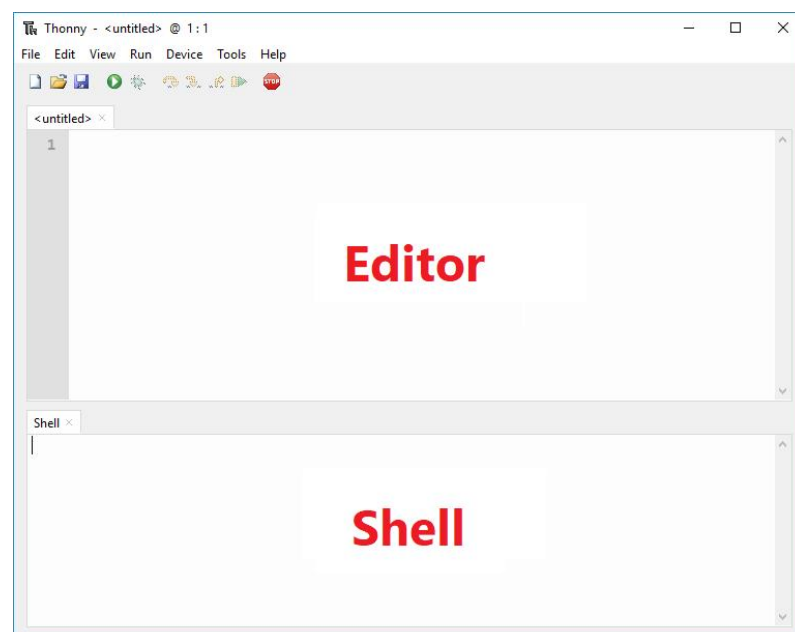
```
>>> print('Hello')
Hello
```

すべてが期待どおりに動作している場合は、Thonny IDE と MicroPython ファームウェアが ESP32 ボードに正常にインストールされたことを意味します。

1.5 Thonny IDE の概要

Thonny IDE を開きます。2 つの異なるセクションがあります:

エディターと MicroPython シェル/ターミナルです:

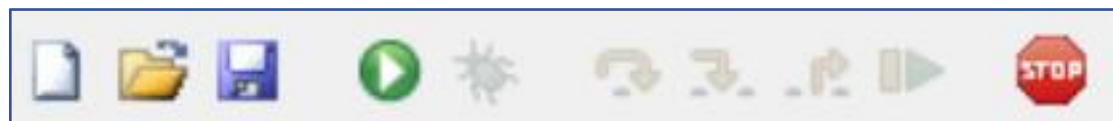


エディターセクションは、コードを記述したり.py ファイルを編集したりする場所です。複数のファイルを開くことができ、エディターはファイルごとに新しいタブを開きます。

MicroPython シェルでは、新しいファイルをアップロードする必要なく、ESP32 ボードで即座に実行するコマンドを入力できます。ターミナルは、実行中のプログラムの状態に関する情報を提供し、アップロードプロセスに関連するエラー、構文エラー、メッセージの表示などを行います。

アイコンについて

Thonny IDE の上部セクションには、プログラミング中によく使用されるメニューバーがいくつかあります：



：新しい Python ファイルを作成します。



：コンピューターから既存のファイルを開くことができます。以前に作業したプログラムに戻る場合に便利です。



：コードを保存します。作成中のコードが失われないように、このメニューを頻繁にクリックする習慣をつけることをお勧めします。



：エディターの現在のページ（表示中）の Python コードを実行します。



：デバッグ中の機能を停止します。

以下の機能は現在、ESP32 C3 用の MicroPython ファームウェアではサポートされておらず、簡単に理解する程度で構いません：



：コードをデバッグできます。



：Python が大きなステップ（次の行またはコードブロックにジャンプ）を実行することを示します。



：Python が小さなステップ（式の各コンポーネントに深く入り込む）を実行することを示します。



：Python がデバッガーから抜け出すことを示します。



：デバッグモードを終了します。

1.6 初めてのコードを実行する - Thonny で現在のコードを実行

1. 開発環境が設定されていることを確認してください。

この時点で、以下が完了している必要があります：

- ✓ 十分に充電された 18650 バッテリーを取り付け、電源スイッチをオンにする。
- ✓ ESP32 ボードをコンピューターに接続する。
- ✓ コンピューターに CH340 ドライバがインストールされている。
- ✓ コンピューターに Thonny IDE がインストールされている。
- ✓ ESP32 ボードに MicroPython ファームウェアが書き込まれている。

ファイルを作成し、ESP32 ボードでコードを実行するプロセスに慣れるために、ESP32 の内蔵ブザーを制御して音を鳴らす簡単な新しいスクリプトをアップロードします。

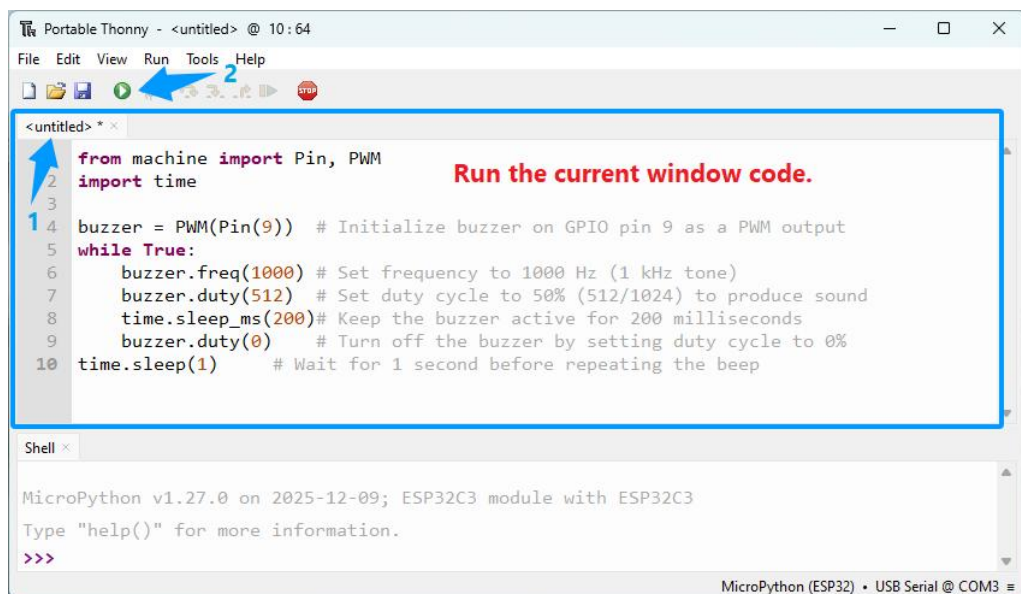
2. コードの実行

Thonny IDE を初めて開くと、エディターには無題のファイルが表示されます。次のスクリプトをそのファイルにコピーします：

```
from machine import Pin, PWM
import time

buzzer = PWM(Pin(9)) # Initialize buzzer on GPIO pin 9 as a PWM output
while True:
    buzzer.freq(1000) # Set frequency to 1000 Hz (1 kHz tone)
    buzzer.duty(512) # Set duty cycle to 50% (512/1024) to produce sound
    time.sleep_ms(200) # Keep the buzzer active for 200 milliseconds
    buzzer.duty(0) # Turn off the buzzer by setting duty cycle to 0%
    time.sleep(1) # Wait for 1 second before repeating the beep
```

ボード上でそのコードを実行するには、Thonny IDE の**実行**アイコンをクリックするだけです。



制御ボードの電源スイッチをオンにしてください。ブザーが音を鳴らします。



プログラムの実行を停止するには、STOP ボタンを押すか、Ctrl+C を押します。

注：

Thonny でファイルを実行するだけでは、ESP32 のファイルシステムに永続的にコピーされません。つまり、コンピューターからボードを取り外して電源を入れ直しても、ファイルシステムに MicroPython ファイルが保存されていないため、何も実行されません。

Thonny IDE の実行機能はコードのテストに便利ですが、ボードに永続的にアップロードするには、ボードのファイルシステムにファイルを作成して保存する必要があります。これについては次に説明します。

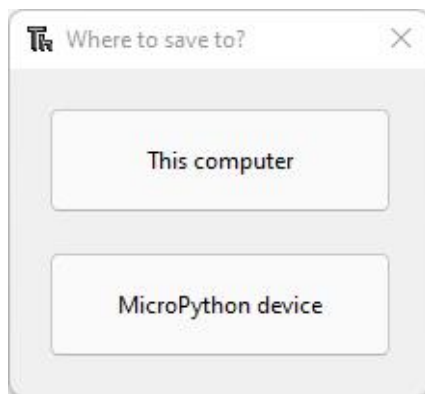
1.7 ESP32 ボードへのコードアップロード - ESP32 でコードを実行

方法 1: main.py として直接保存/アップロード

1) ブザーコードを実行した後、停止アイコンをクリックして実行を停止します。次に、保存アイコンをクリックするか、File > Save as... を選択してファイルを保存します。



2) 次に、「MicroPython device」を選択します

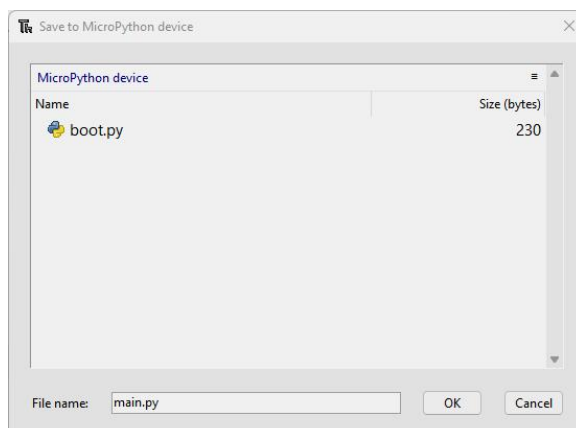


注：

Q：Thonny のファイルパネルに「**MicroPython Device**」が表示されません。

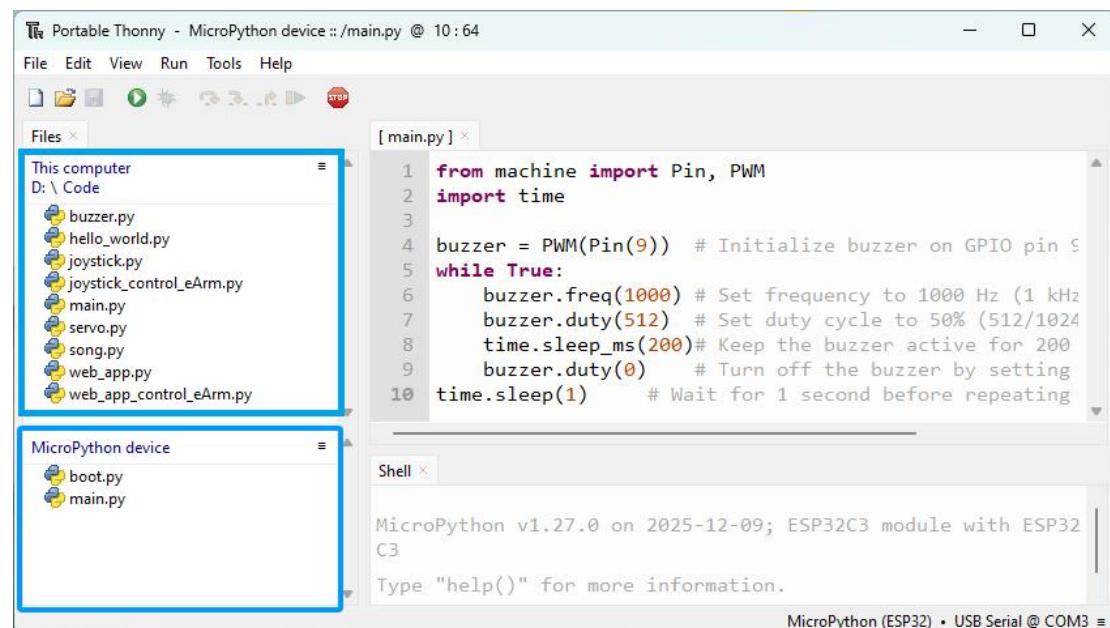
A：コンピューターの USB ポートが制御ボードに接続されるたびにこの現象が発生します。これは Thonny に ESP32 デバイスの接続を自動検出する機能がないためです。**STOP** ボタンをクリックするか、Thonny 右下の「**MicroPython (ESP32) - USB Serial @ COMx**」を選択し直すと解決します。

3) ファイル名を main.py とします。そうしないと、ESP32 で自動的に実行されません。



表示されるウィンドウには、ボードのファイルシステムに現在保存されているファイルが表示されます。**boot.py** というファイルがあるはずです。このファイルは MicroPython ファームウェアを書き込むときにデフォルトで作成されます。

- 4) 2つのファイルパネルが表示されます：「This computer」（ローカルファイル）と「MicroPython device」（接続された ESP32 の内部ストレージ）です。



- 5) ボードの RESET ボタンを押して、ボードを再起動し、コードの実行を開始します。
6) ESP32 ボードのブザーが毎秒鳴るはずですが。

ファイルを保存した後、ボードを取り外して再び電源を入れることができます。ESP32 ボードのブザーが電源投入時に自動的に音を鳴らすことに注目してください。これは、main.py ファイルが MicroPython device に正常にアップロードされ、そのファイルが自動的に実行されていることを意味します。

重要： ファイル名を main.py とすると、ESP32 は起動時にそのファイルを自動的に実行します。別の名前では保存すると、ボードのファイルシステムには保存されますが、起動時に自動的に実行されることはありません。

boot.py は無視できますか？

はい、boot.py ファイルは安全に無視できます。削除する必要はありません。ほぼすべての初心者および中級者の ESP32 MicroPython プロジェクト（例：LED 点滅、センサーデータ読み取り、基本的な WiFi 接続、簡単なシリアル通信）では、ESP32 の MicroPython ランタイムは完全に正常に動作し、すべてのプロジェクトロジックを直接 main.py に記述できます（ピンやセンサーのセットアップなどの簡単な初期化手順を含む）。boot.py は、コード構成をよりクリーンにするために、システムレベルの初期化をメインのアプリケーションロジックから分離する必要がある場合にのみ使用します。

要約すると：

boot.py：「システム環境」を準備する

main.py：「実際のプロジェクト」を実行する

REPL：「デバッグツール」

この3つが連携して ESP32 MicroPython の完全な実行ロジックを形成し、boot.py はシステムレベルの構成のためのオプションですが強力な補足機能です。

方法 2: main.py が他のファイルを読み出す

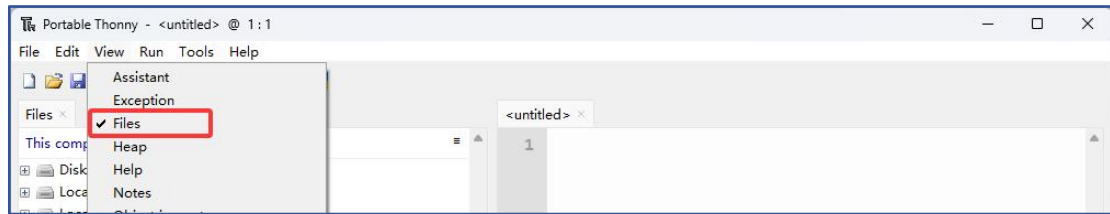
2つの方法の比較

方法	操作手順	利点	欠点	適用シナリオ
① main.py として直接保存 (直接アップロード)	1. チュートリアルコードを開く 2. 名前を付けて保存 → デバイスを選択 3. main.py という名前を付ける	すべてのコードが main.py ファイルに記述されており、シンプルな単一ファイルプロジェクトに適している	1. 結合度が高く、デバッグが不便 2. 修正のたびにコード全体を再アップロードする必要がある 3. モジュール化されていない	単一ファイルの実験、一度きりのテスト
② main.py が他のファイルを読み出す	1. 機能コード (例 :buzzer.py) をデバイスにアップロード 2. 新しい main.py ファイルを作成し、main.py に他のファイルを読み出すコードを記述 3. main.py を ESP32 デバイスにアップロード	1. モジュール化され、保守が容易 2. サブモジュールの独立したデバッグが可能 3. コードの再利用をサポート	ファイル管理の手順がやや多い	マルチモジュールプロジェクト、長期的な開発

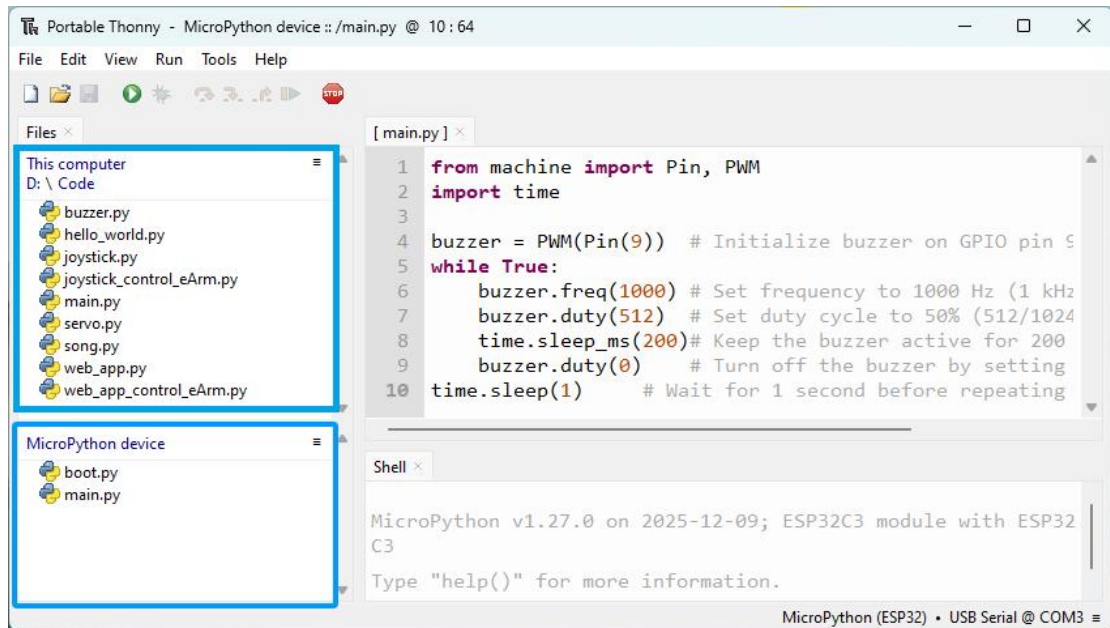
2 番目の方法は、より専門的で柔軟性が高く、組み込み開発の推奨プラクティスに沿っているため、選択することをお勧めします。

チュートリアルパッケージには、さまざまな機能の xxx.py ファイルと main.py が用意されています。main.py のコードは、「MicroPython device」にある main.py と boot.py 以外の xxx.py ファイルを読み出します。

1) メニューバーで「View」をクリックし、ドロップダウンから「Files」を選択します。



2) 2つのファイルパネルが表示されます：「This computer」（ローカルファイル）と「MicroPython device」（接続された ESP32 の内部ストレージ）です。

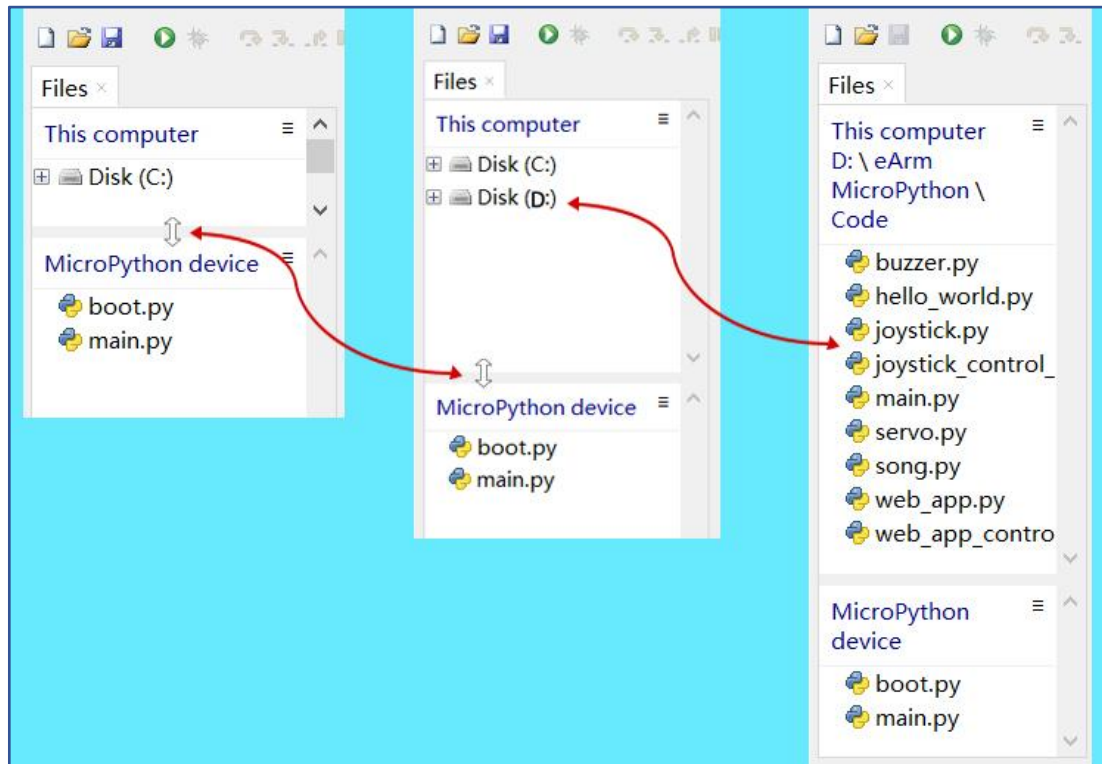


注：

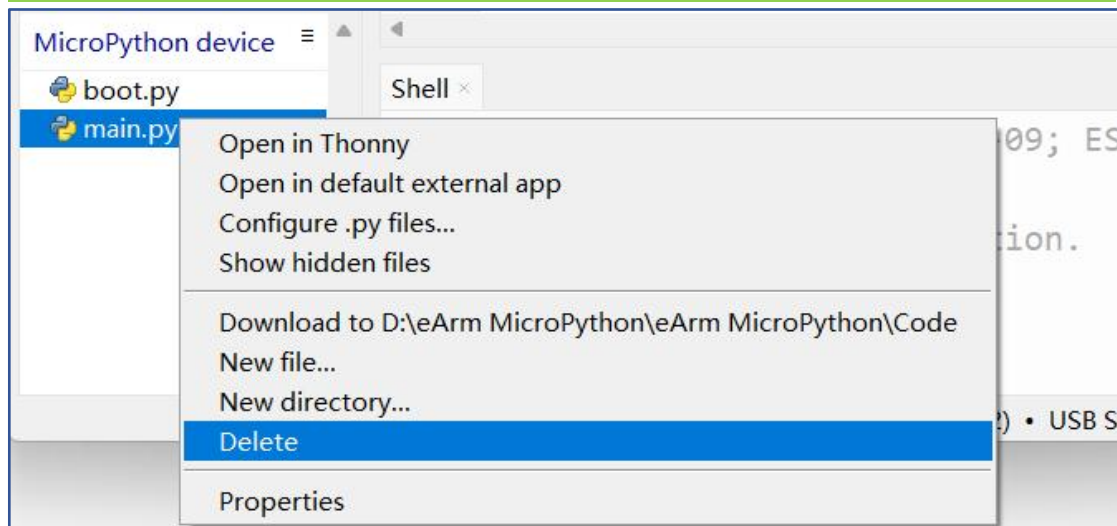
Q：Thonny のファイルパネルに「MicroPython Device」が表示されません。

A：コンピューターの USB ポートが制御ボードに接続されるたびにこの現象が発生します。これは Thonny に ESP32 デバイスの接続を自動検出する機能がないためです。STOP ボタンをクリックするか、Thonny 右下の「MicroPython (ESP32) - USB Serial @ COMx」を選択し直すと解決します。

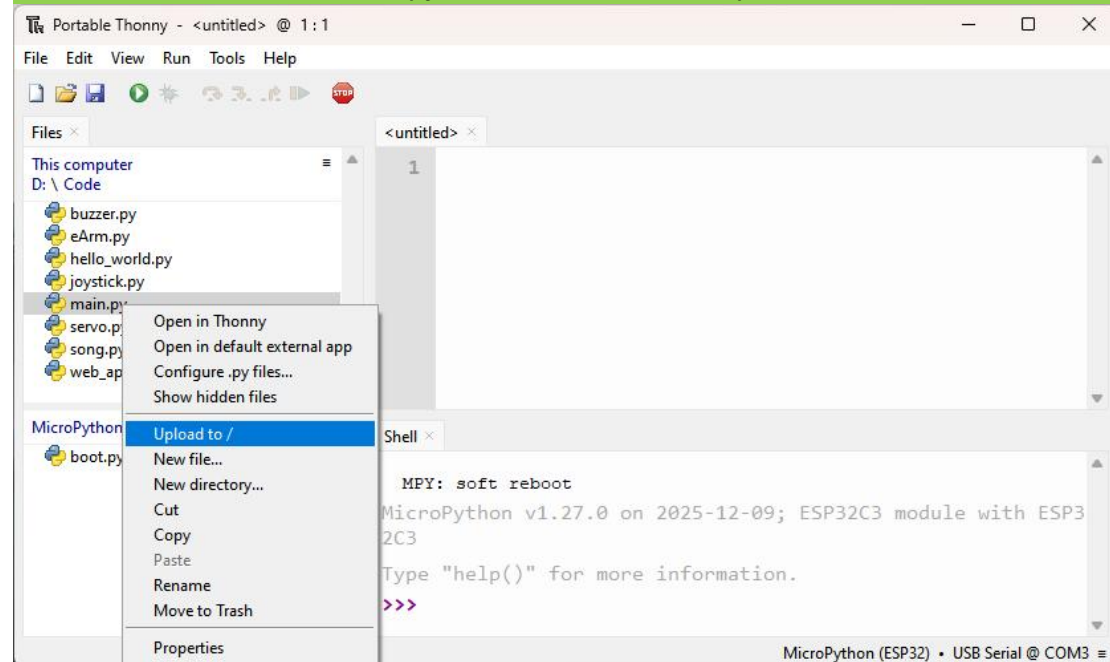
3)ナビゲーションバーをスクロールダウンして、フォルダディレクトリビューを展開します。ダウンロードしたチュートリアルパッケージのコードが保存されているディレクトリに移動します。



4) 「MicroPython device」で以前にアップロードしたブザーを鳴らすための「main.py」を右クリックし、ESP32-C3 のルートディレクトリから削除します。

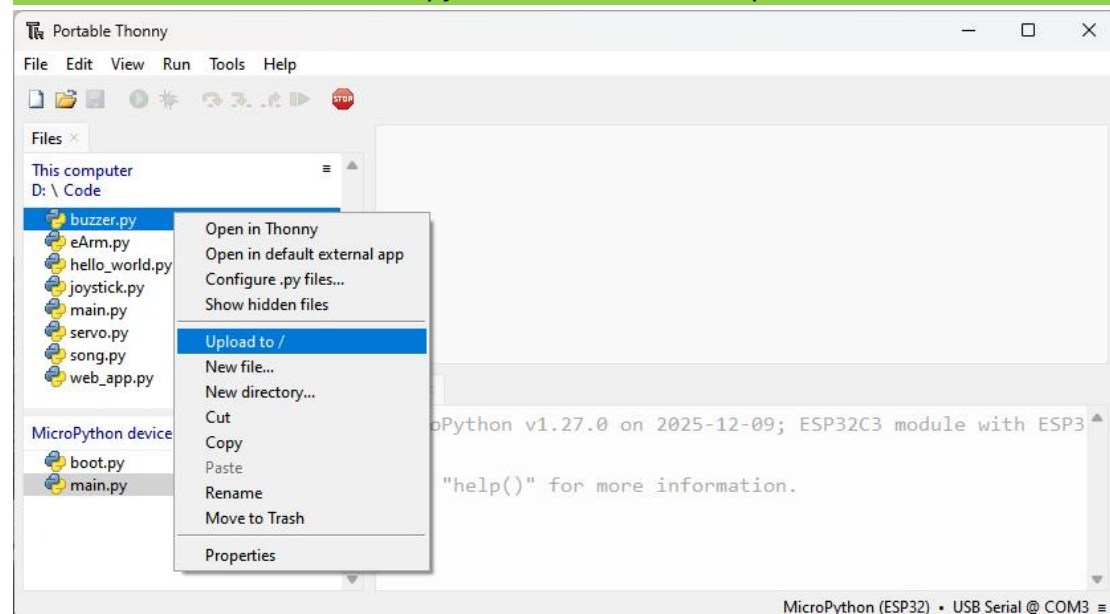


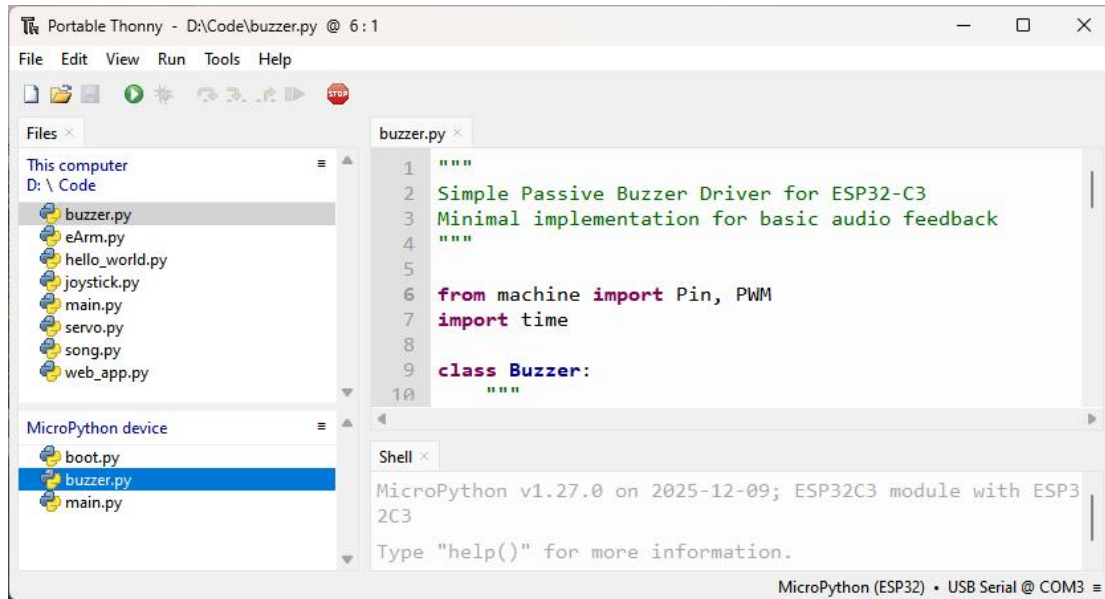
5) ローカルファイルの「main.py」を右クリックし、「Upload to/」をクリックします。



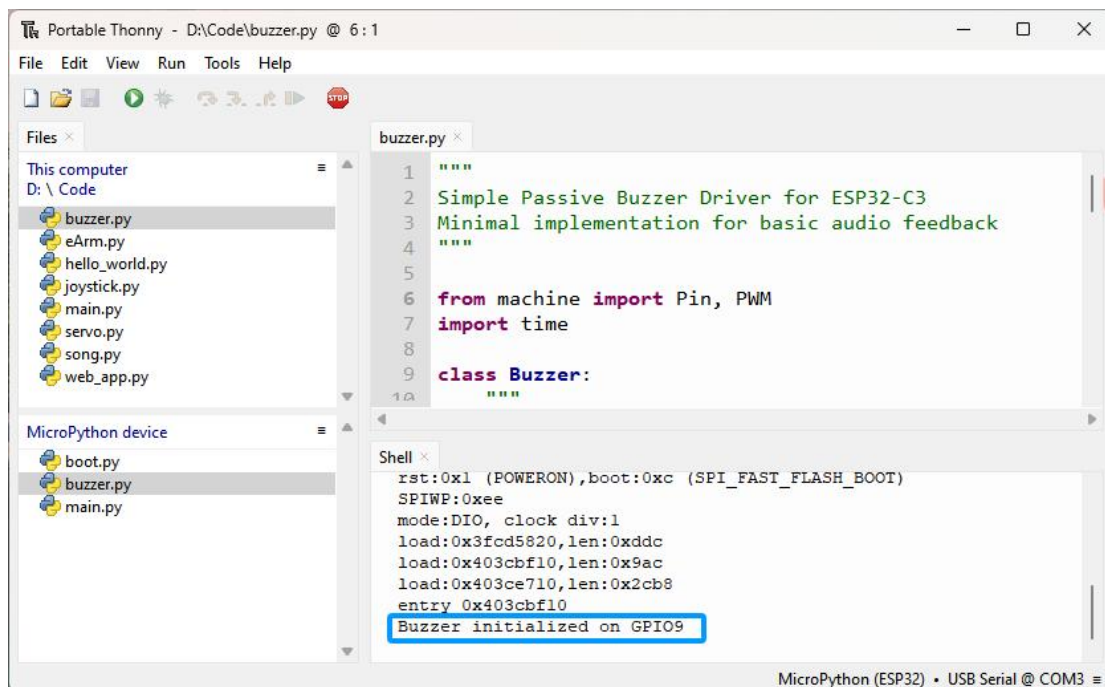
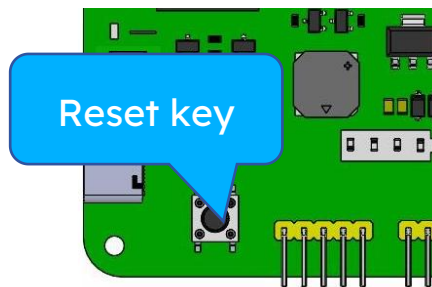
重要：「MicroPython device」にファイルをアップロードする前に、オンラインでの実行を停止する必要があります！（Thonnyの「STOP」メニューをクリック）

6) ローカルファイルの「buzzer.py」を右クリックし、「Upload to/」をクリックします

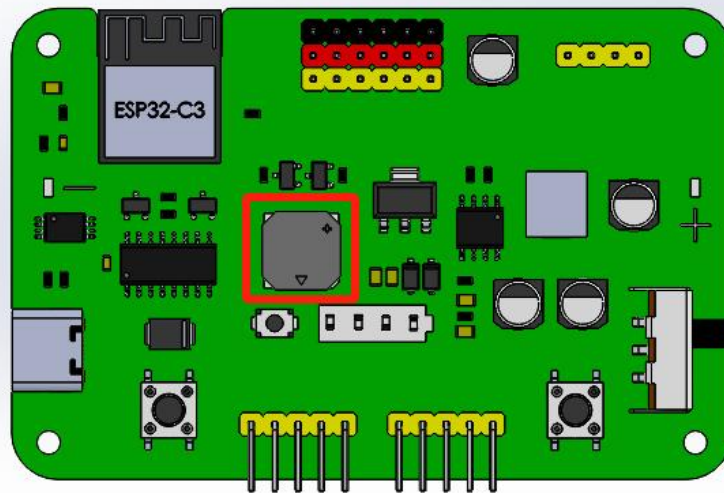




7) esp32 ボードのリセットボタンを押すと、コードが実行されます



8) 電源スイッチをオンにすると、ESP32-C3 ボードのブザーが鳴り続けます

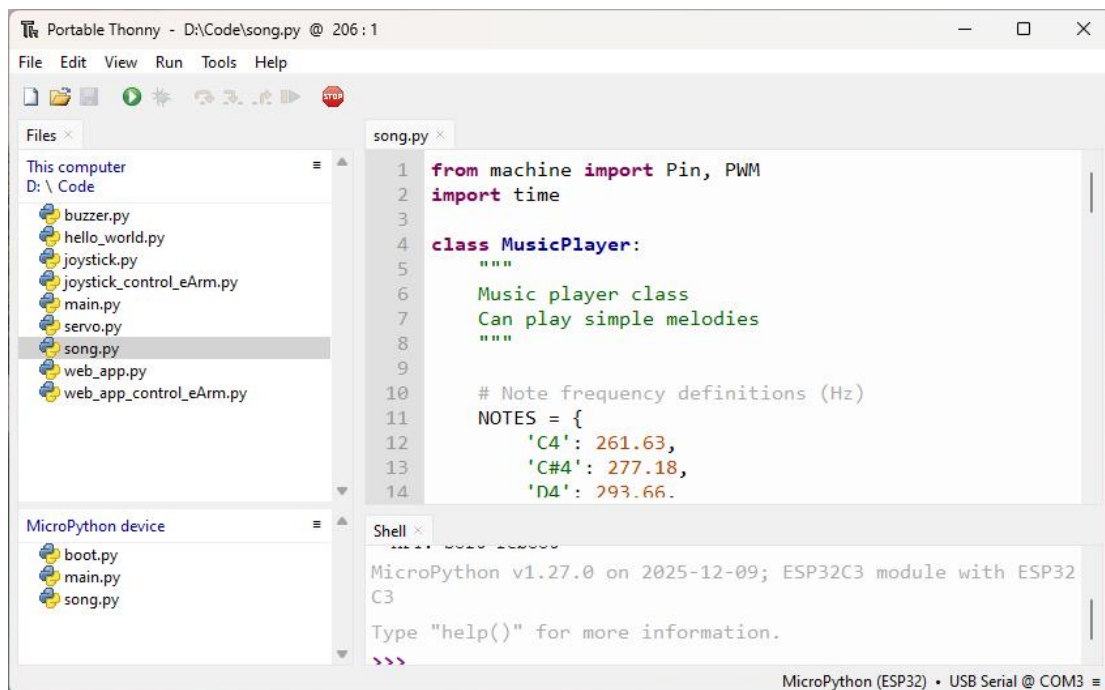


重要： 提供されている main.py ファイルは、一度に 1 つの非 main.py コードファイルのみを呼び出すように設計されています。複数のファイルを同時にアップロードすると、リセット時にシステムが誤動作する可能性があります（意図したファイルではなく、異なるファイルがランダムに実行される可能性があります）

1.8 コード例

1.8.1 曲を再生する

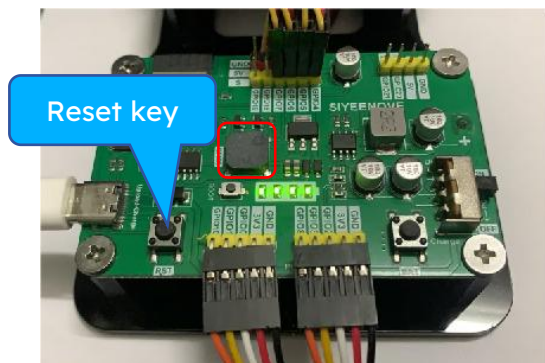
- 1 MicroPython デバイスから以前にアップロードした buzzer.py を削除します。提供されている main.py は一度に 1 つのモジュールファイルのみを呼び出すように設計されているためです。
- 2 コードが実行中の場合は、Stop ボタンをクリックします。次に、song.py を右クリックし、「Upload to /」を選択します。
- 3 song.py ファイルが MicroPython デバイスのファイルリストに表示されます。
- 4 制御ボードの電源スイッチが ON で、バッテリーに十分な充電があることを確認します。
- 5 esp32 制御ボードのリセットボタンを押して再起動します。ボードのブザーが音楽を再生し始めます。
- 6 コードを表示するには :song.py に移動し、ファイルをダブルクリックして Thonny で開きます。



注：

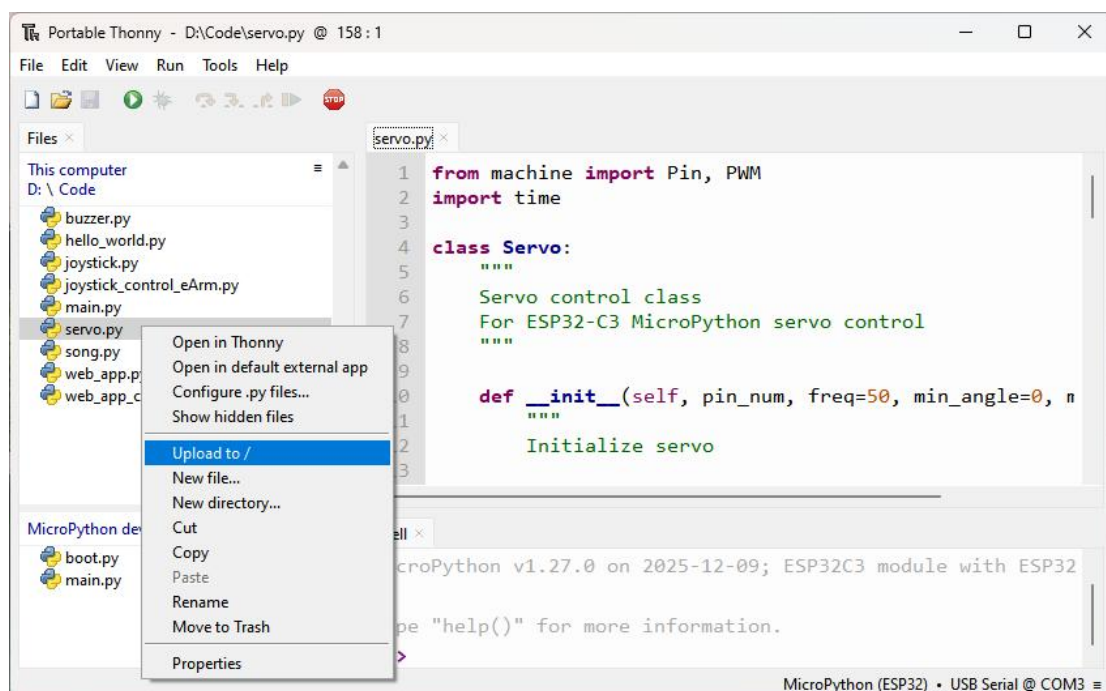
Q：Thonny のファイルパネルに「MicroPython Device」が表示されません。

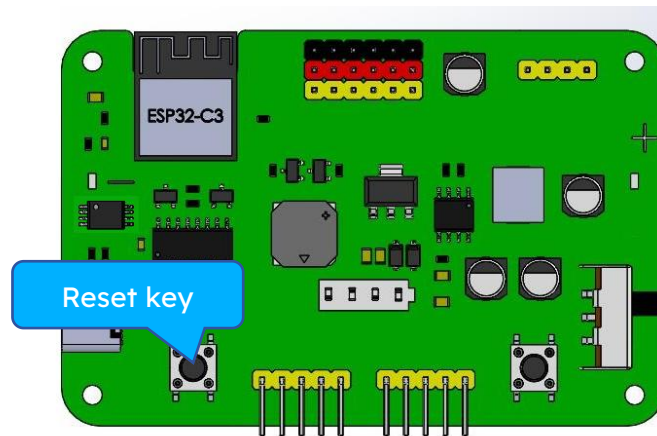
A：コンピューターの USB ポートが制御ボードに接続されるたびにこの現象が発生します。これは Thonny に ESP32 デバイスの接続を自動検出する機能がないためです。STOP ボタンをクリックするか、Thonny 右下の「MicroPython (ESP32) - USB Serial @ COMx」を選択し直すと解決します。



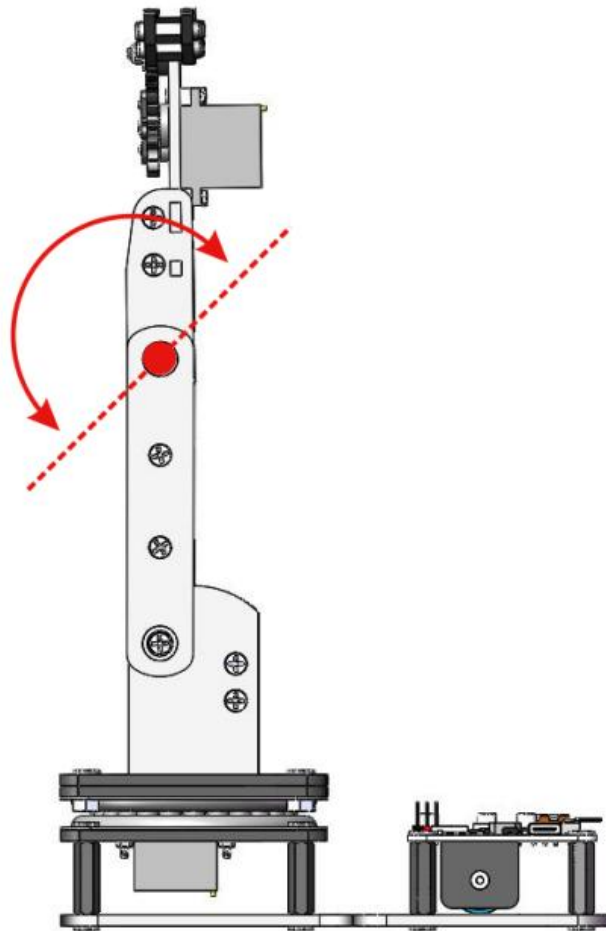
1.8.2 サーボを動かす

- 1 MicroPython デバイスから以前にアップロードした `song.py` を削除します。提供されている `main.py` は一度に 1 つのモジュールファイルのみを呼び出すように設計されているためです。
- 2 コードが実行中の場合は、Stop ボタンをクリックします。次に、`servo.py` を右クリックし、「Upload to /」を選択します。
- 3 `servo.py` ファイルが MicroPython デバイスのファイルリストに表示されます。
- 4 制御ボードの電源スイッチが ON で、バッテリーに十分な充電があることを確認します。
- 5 esp32 制御ボードのリセットボタンを押して再起動します。eArm のサーボ C が 0-180 度、180-0 度の周期で揺れ動きます。
- 6 コードを表示するには `:servo.py` に移動し、ファイルをダブルクリックして Thonny で開きます。



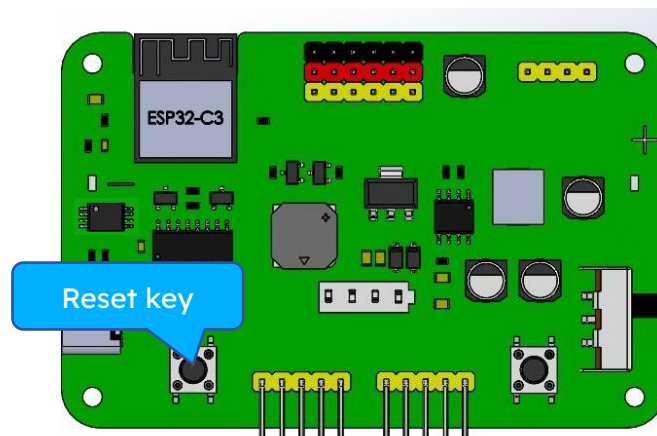
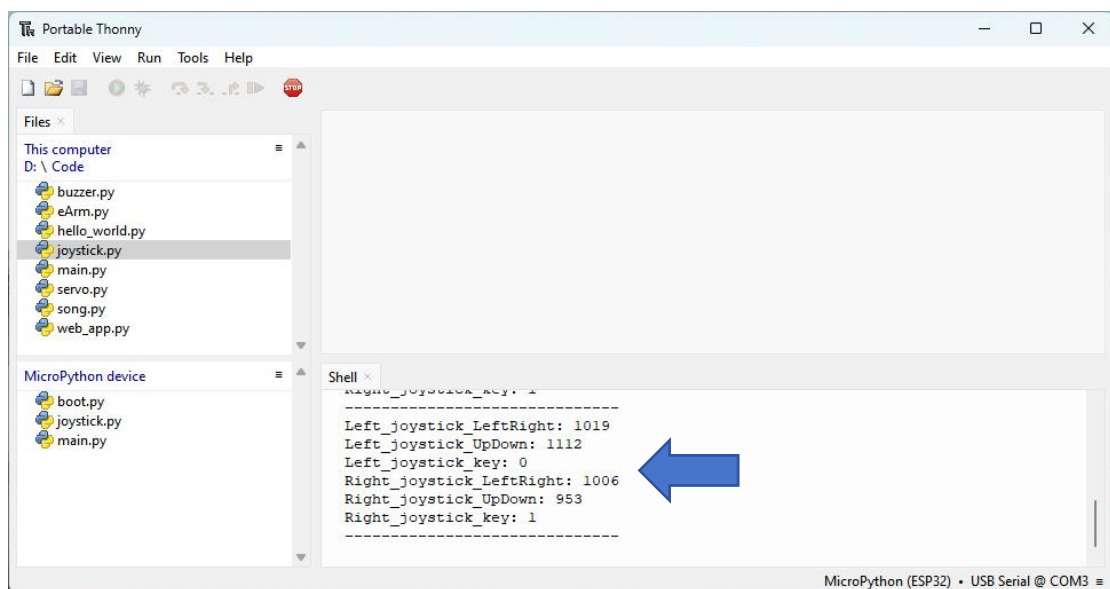


コードが正常にアップロードされると、eArm のサーボ C が 0-180 度、180-0 度の周期で揺れ動きます：



1.8.3 ジョイスティックの値を読み取る

- 1 MicroPython デバイスから以前にアップロードした servo.py を削除します。提供されている main.py は一度に 1 つのモジュールファイルのみを呼び出すように設計されているためです。
- 2 コードが実行中の場合は、Stop ボタンをクリックします。次に、joystick.py を右クリックし、「Upload to /」を選択します。
- 3 joystick.py ファイルが MicroPython デバイスのファイルリストに表示されます。
- 4 制御ボードの電源スイッチが ON で、バッテリーに十分な充電があることを確認します。
- 5 esp32 制御ボードのリセットボタンを押して再起動します。ジョイスティックを左、右、上、下に動かすと、シェルにジョイスティックの値が表示されます。
- 6 コードを表示するには：joystick.py に移動し、ファイルをダブルクリックして Thonny で開きます。



1.8.4 ジョイスティックで eArm を制御する(記録機能付き)

MicroPython デバイスから以前にアップロードした joystick.py を削除します。

- 1 提供されている main.py は一度に 1 つのモジュールファイルのみを呼び出すように設計されているためです。

- 2 コードが実行中の場合は、Stop ボタンをクリックします。次に、joystick_control_eArm.py を右クリックし、「Upload to /」を選択します。
- 3 joystick_control_eArm.py ファイルが MicroPython デバイスのファイルリストに表示されます。

- 4 制御ボードの電源スイッチが ON で、バッテリーに十分な充電があることを確認します。

- 5 esp32 制御ボードのリセットボタンを押して再起動します。ジョイスティックを押したり動かしたりしてロボットアームを制御できます。

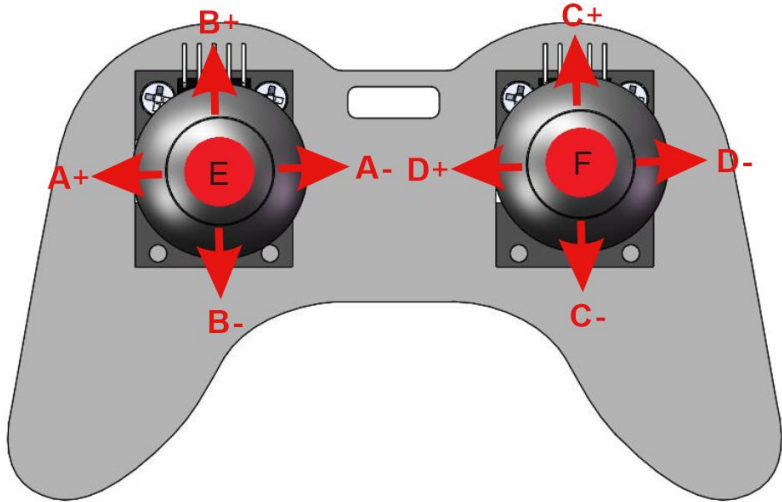
- 6 コードを表示するには：joystick_control_eArm.py に移動し、ファイルをダブルクリックして Thonny で開きます。

The screenshot shows the Thonny IDE interface. On the left, the 'Files' pane shows the file structure of the 'This computer' (D:\Code) and the 'MicroPython device'. The 'joystick_control_eArm.py' file is highlighted in the 'This computer' list. The main editor window displays the code for 'joystick_control_eArm.py'. The code defines a 'JoyStick' class with methods for initialization, attaching pins, and eliminating jitter. The 'Shell' pane at the bottom shows the output of the MicroPython interpreter, indicating a soft reboot and the version of the MicroPython firmware.

```
1 # joystick_control_eArm.py
2 # MicroPython version for ESP32-C3
3 # This code applies to siyeenove mechanical arm
4 # Through this link you can download the source code:
5 # https://github.com/siyeenove
6 # Company web site:
7 # https://siyeenove.com/
8 from machine import Pin, ADC, PWM
9 import time
10
11 class JoyStick:
12     """Joystick class"""
13     def __init__(self):
14         self.pin_x = None
15         self.pin_y = None
16         self.pin_z = None
17         self.adc_x = None
18         self.adc_y = None
19         self.btn_z = None
20         self.buf = [0] * 20
21
22     def attach(self, x_pin, y_pin, z_pin=None):
23         """Attach joystick pins"""
24         self.pin_x = x_pin
25         self.pin_y = y_pin
26         self.adc_x = ADC(Pin(x_pin))
27         self.adc_y = ADC(Pin(y_pin))
28         self.adc_x.atten(ADC.ATTN_11DB) # 0-3.3V range
29         self.adc_y.atten(ADC.ATTN_11DB)
30
31         if z_pin is not None:
32             self.pin_z = z_pin
33             self.btn_z = Pin(z_pin, Pin.IN, Pin.PULL_UP)
34
35     def _eliminate_jitter(self):
```

MPY: soft reboot
MicroPython v1.27.0 on 2025-12-09; ESP32C3 module with ESP32C3
Type "help()" for more information.
>>>

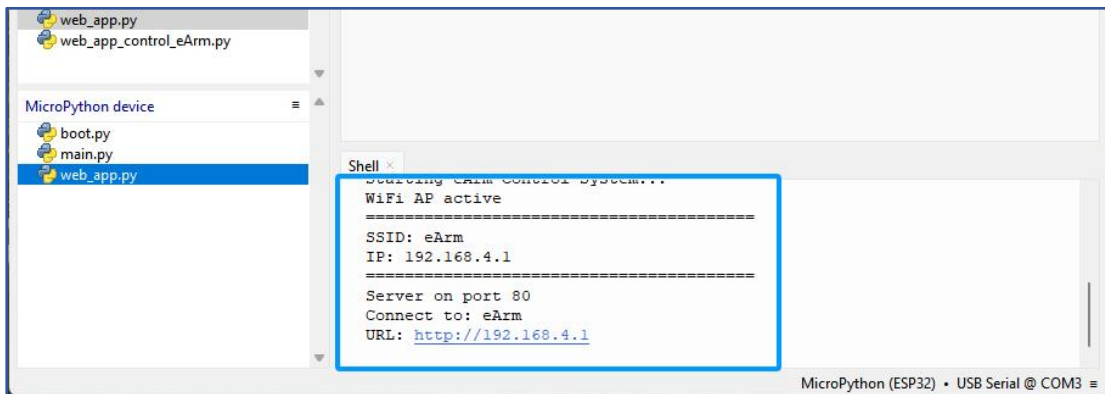
ジョイスティック制御ロボットアームのデモンストレーション：

	
A+ : A servo を制御し、ロボットアームを左に回転させます。	A- : A servo を制御し、ロボットアームを右に回転させます。
B+ : B servo を制御し、ロボットアームの大アームを前へ振り出します。	B- : B servo を制御し、ロボットアームの大アームを後ろへ振り出します。
C+ : C servo を制御し、ロボットアームの小アームを前へ振り出します。	C- : C servo を制御し、ロボットアームの小アームを後ろへ振り出します。
D+ : D servo を制御し、ロボットアームのグリッパーを開きます（緩めます）。	D- : D servo を制御し、ロボットアームのグリッパーを閉じます（締めます）。
E : 記録機能 - E ボタンを 1 回押すと、1 つの動作を記録します（最大 20 個の動作を順番に記録可能）。押すたびにブザーが短く 1 回鳴ります。記録数が最大に達すると、ブザーが長く 1 回鳴ります。	F : 再生 — 動作シーケンスの記録が完了したら、F ボタンを 1 回押すと、記録された動作を連続で繰り返し再生（ループ再生）します。ブザーが 1 回鳴り、機械臂が記録された動作をループで繰り返します。 ループを終了するには、F ボタンを長押ししてください。ブザーが長く 1 回鳴り、再生が停止したことをお知らせします。

1.8.5 Web アプリの値を読み取る

- MicroPython デバイスから以前にアップロードした joystick_control_eArm.py を削除します。提供されている main.py は一度に 1 つのモジュールファイルのみを呼び出すように設計されているためです。
- コードが実行中の場合は、Stop ボタンをクリックします。次に、web_app.py を右クリックし、「Upload to /」を選択します。
- web_app.py ファイルが MicroPython デバイスのファイルリストに表示されます。制御ボードの電源スイッチが ON で、バッテリーに十分な充電があることを確認します。
- esp32 制御ボードのリセットボタンを押して再起動します。シェルに esp32 のネットワークサービス起動情報が表示されます。ESP32 の WiFi に接続したら、ブラウザで Web アプリを開きます。各ボタンをクリックすると、その値がシェルに記録されます。
- コードを表示するには：web_app.py に移動し、ファイルをダブルクリックして Thonny で開きます。

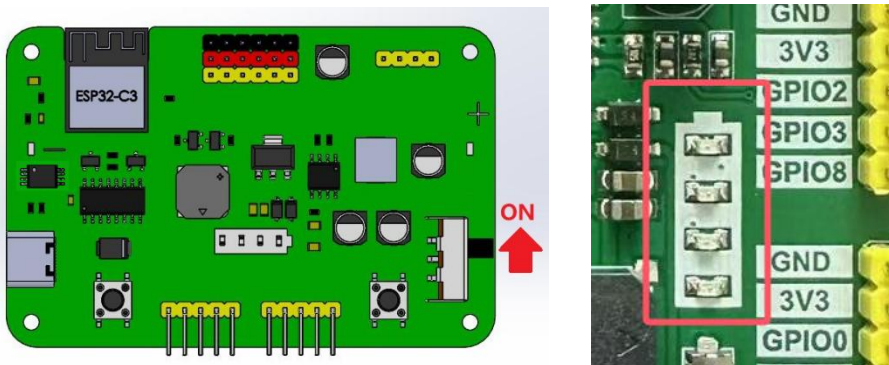
コードをアップロードすると、シェルに WiFi 情報が表示されます。



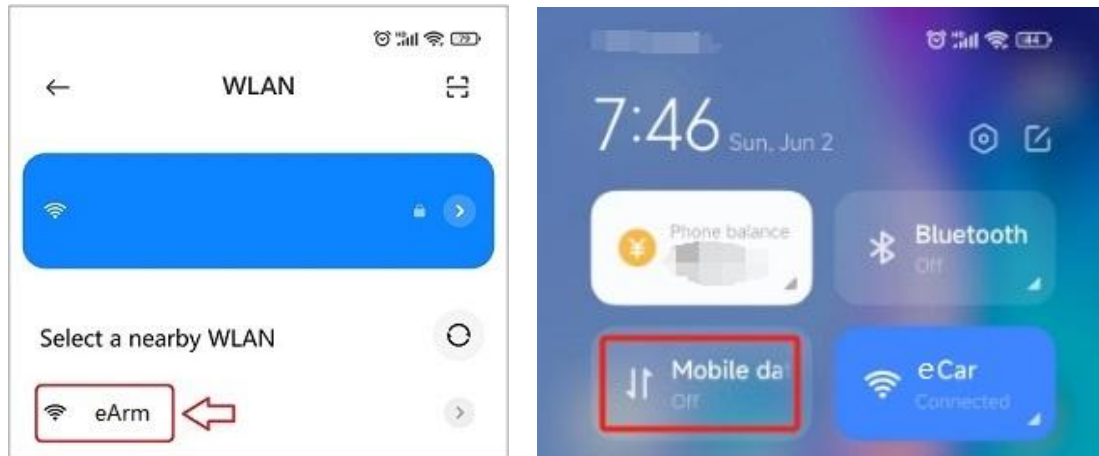
WiFi に接続するには、以下の手順に従ってください。

1) eArm の電源スイッチを ON の位置にします。

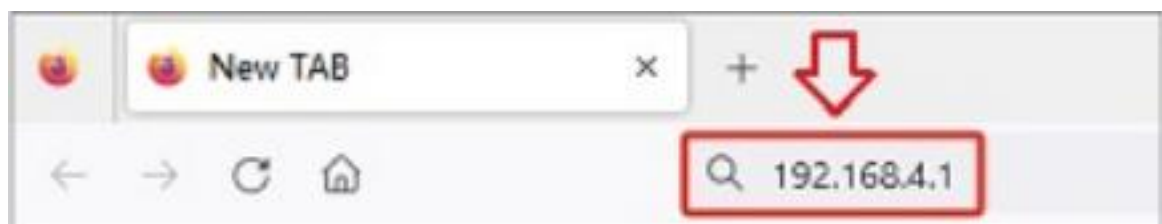
- ：18650 リチウム電池が取り付けられていることを確認します。
- ：バッテリーインジケータが 3 メモリ以上表示されていることを確認します。



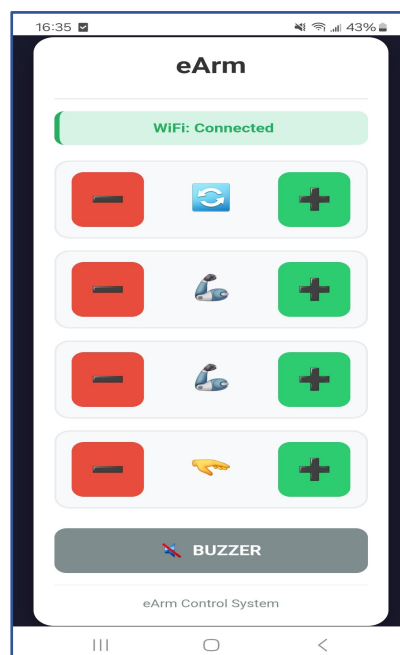
- 2) スマートフォンの Wi-Fi をオンにし、「eArm」というネットワークに接続します。
- ：Wi-Fi に接続すると、「ネットワーク接続に失敗しました」というメッセージが表示される場合がありますが、無視してください。
 - ：スマートフォンの設定でモバイルデータをオフにします。
 - ：これにより、Web アプリへの安定した接続が保証されます。



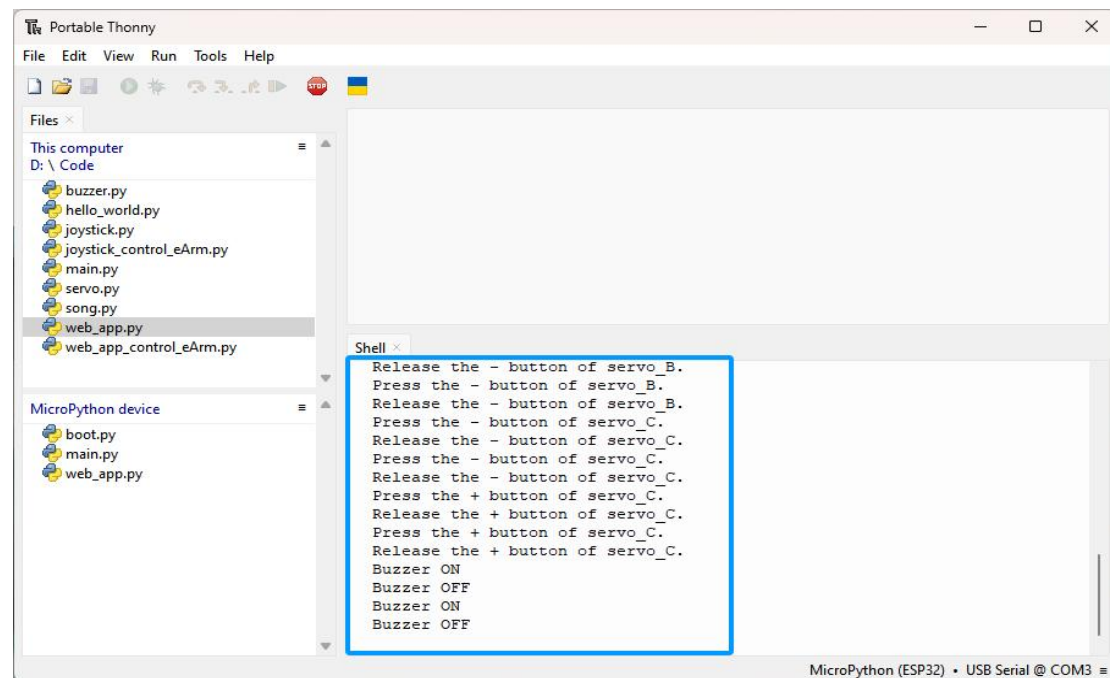
- 3) スマートフォンで Web ブラウザを開きます。
- 4) アドレスバーに「192.168.4.1」と入力します。
- 5) Enter を押して接続します。



WiFi 接続が成功すると、ブラウザにアプリ制御インターフェースが表示されます。

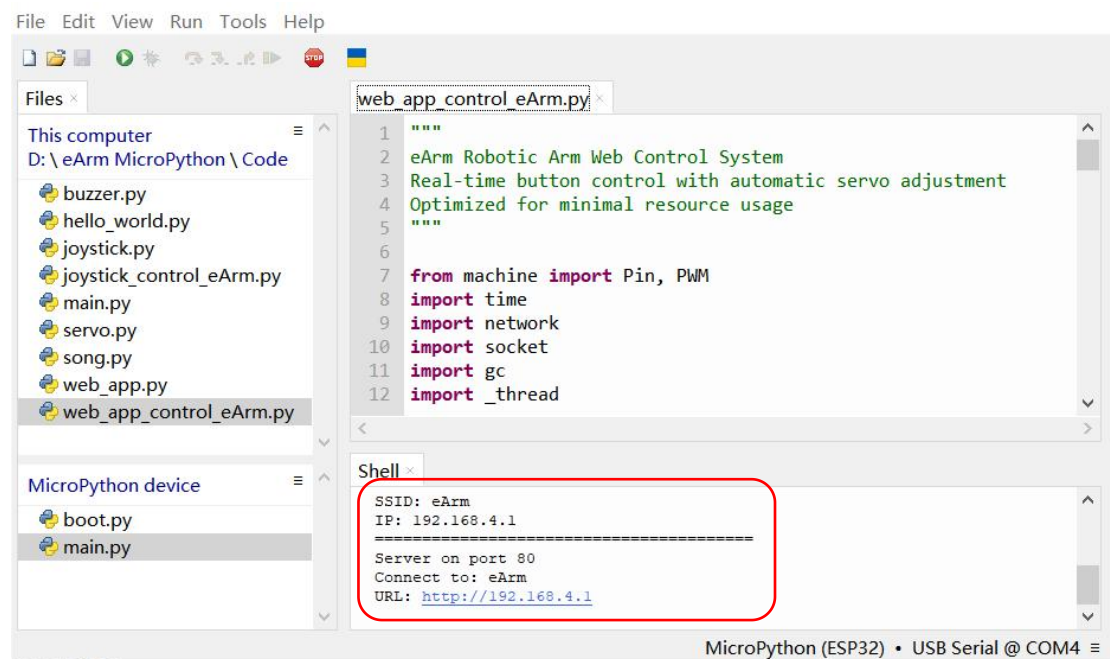
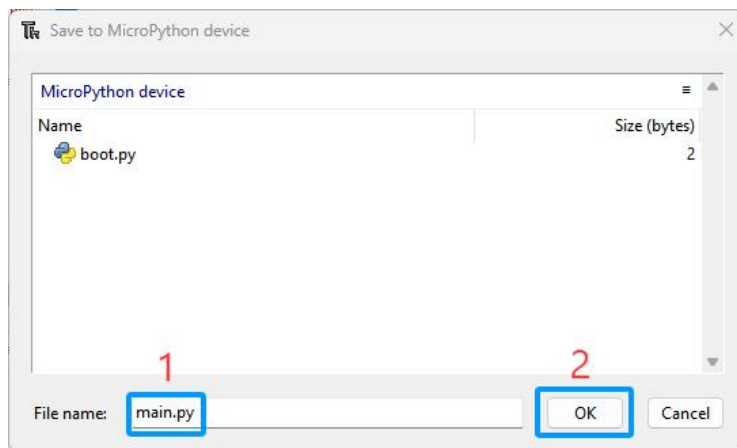
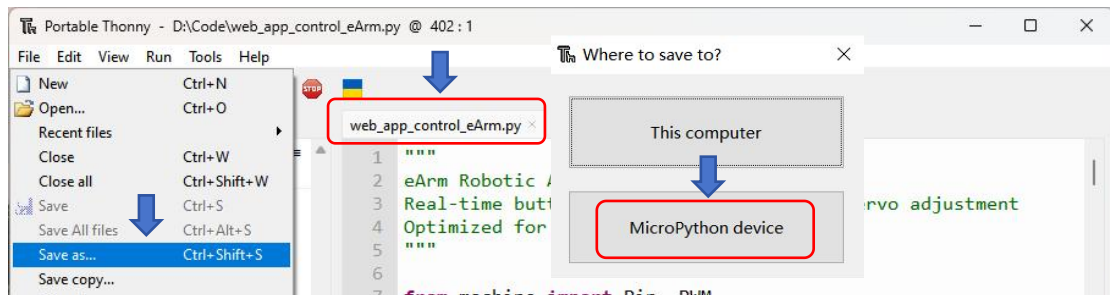
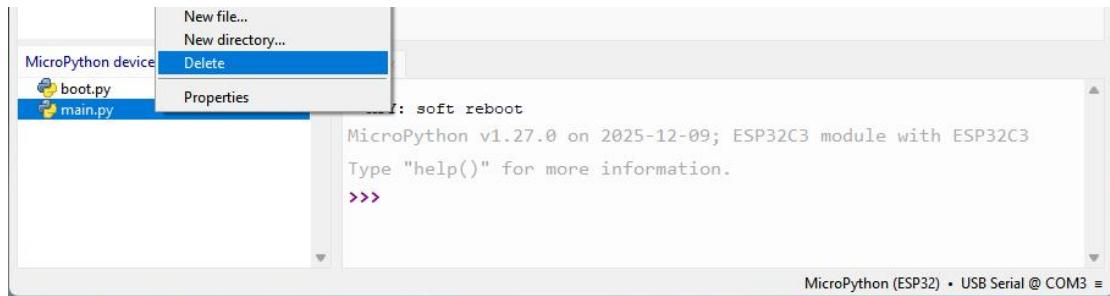


Web アプリのボタンをクリックすると、シェルにキー情報が表示されます。

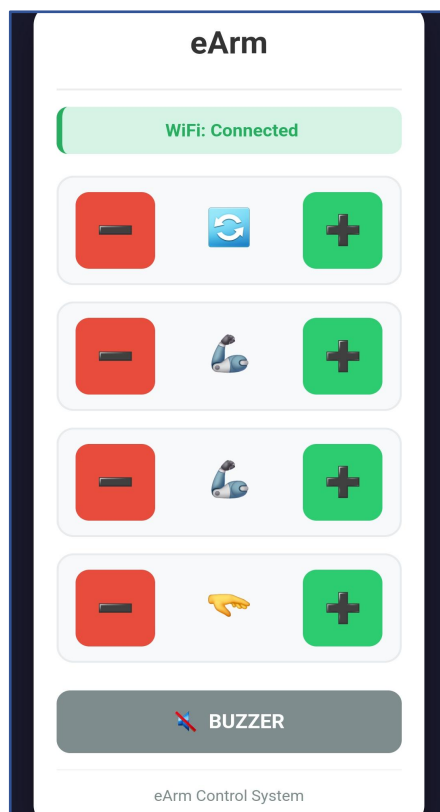


1.8.6 Web アプリで eArm ロボットを制御する

- 1 MicroPython デバイスから以前にアップロードした web_app.py と main.py を削除します。
- 2 この最終レッスンでは、コードを直接 main.py として保存し、MicroPython デバイスにアップロードします。メモリの過負荷を避け、スムーズな動作を確保します。
- 3 web_app_control_eArm.py に移動し、ファイルをダブルクリックして Thonny で開きます。
- 4 File > Save as... をクリックし、MicroPython デバイスを選択します。ファイル名を main.py とし、OK をクリックして続行します。
- 5 制御ボードの電源スイッチが ON で、バッテリーに十分な充電があることを確認します。
- 6 esp32 制御ボードのリセットボタンを押して再起動します。シェルに esp32 のネットワークサービス起動情報が表示されます。ESP32 の WiFi に接続したら、ブラウザで Web アプリを開いてロボットアームを制御できます。



前のレッスンの方法を使用して ESP32 Wi-Fi に接続するか、[こちらをクリックしてください](#)。



		ロボットアーム左回転
		ロボットアーム右回転
		リアアーム上昇
		リアアーム下降
		フロントアーム上昇
		フロントアーム下降
		グリッパー開く
		グリッパー閉じる
		ブザー ON
		ブザー OFF

重要！グリッパーの重要なメンテナンス（サーボ保護）

負荷のかかった状態で物体を握り続けると、サーボモーター（特にグリッパーを駆動するD軸）に大きな負担がかかり、熱を発生させます。そのため、負荷をかけた連続的なグリッップ動作は40秒以内に制限することを強くお勧めします。グリッップ操作が完了したら、グリッパーを開いてすぐに物体を放し、モーターを冷却・休息させてください。この時間を超えると、モーターの過熱や焼損の主な原因となります。

当社の最新の最適化されたファームウェア/コードには保護機能が含まれています。グリッパーサーボが40秒以内に新しいコマンドを受信しない場合、自動的に力を解除（PWM信号の送信を停止）して過熱を防ぎます。

ベストプラクティス：グリッパーを積極的に制御してください。物体を操作していないときは、グリッパーを開くか、リラックスした状態を維持するコマンドを送信してください。

2

ファームウェアの書き込み

ファームウェアファイル（.bin や.hex 形式）としてデバイスの「工場リセットプログラム」を直接ユーザーに提供し、Thonny 開発環境を構築せずにデバイスに書き込むことで復元プロセスを完了できます。

1. 従来の方法の問題点

通常、Thonny 経由でデバイスを復元するには、ユーザーは以下が必要でした：

- ▶Thonny IDE または関連開発ツールのインストール
- ▶ソースコード（.py ファイル）のダウンロード（追加ライブラリの設定や依存関係解決が必要な場合あり）
- ▶コンパイルとデバイスへのアップロード

このプロセスは非技術者にとってハードルが高く、環境関連の問題で失敗する可能性があります。

2. ファームウェア直接提供の利点

ユーザープロセスの簡素化：

- ▶プリコンパイル済みファームウェアファイル（例：firmware.bin）を提供し、シリアルフラッシャーなどの簡単なツールで 1 ステップで書き込み可能
- ▶Thonny IDE のインストール、コンパイルエラーの解決、ライブラリ競合の管理が不要

使用例：

- ▶不調なデバイスを工場出荷状態に復元
- ▶複数デバイスへの同一ファームウェアの一括書き込み（効率化）

2.1 書き込みツール

Burning Tool は次のフォルダに保存されています:



また、公式ウェブサイトから最新バージョンの Burning Tool をダウンロードすることもできます:

<https://www.espressif.com/en/support/download/other-tools>

Flash Download Tools

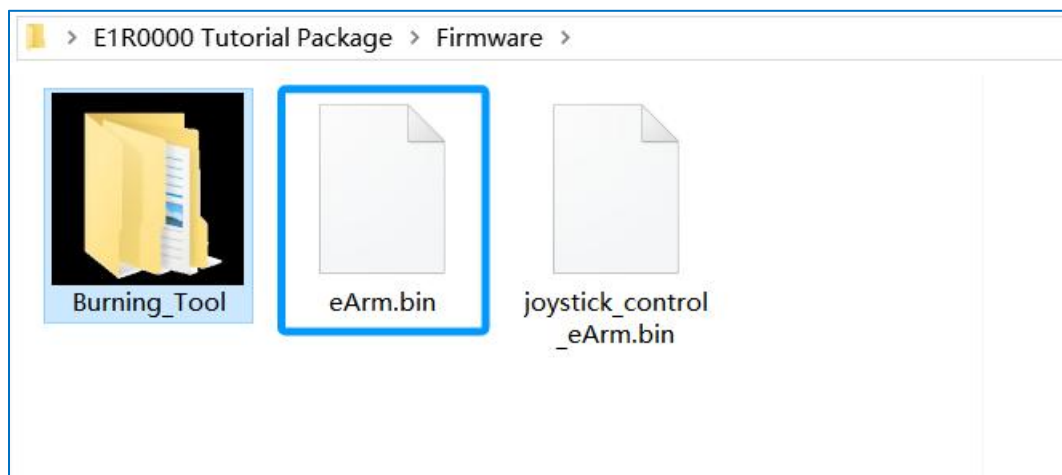
Expand all +

Please refer to Flash Download Tool User Guide to download this tool.

Title	Platform	Version	Release Date ▾	Download
+ Flash Download Tools	HTML	latest	2024.12.18	

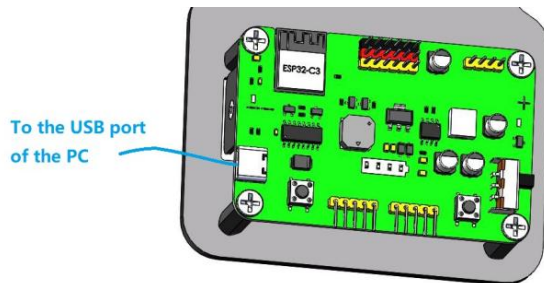
2.2 eArm ファームウェア（工場出荷時ファームウェア）

このファームウェアは、工場出荷時のロボットアームと全く同じ機能を備えています。工場出荷時の機能が失われた場合、このファームウェアを使用して機能を復元できます。以下のファイルに保存されています:

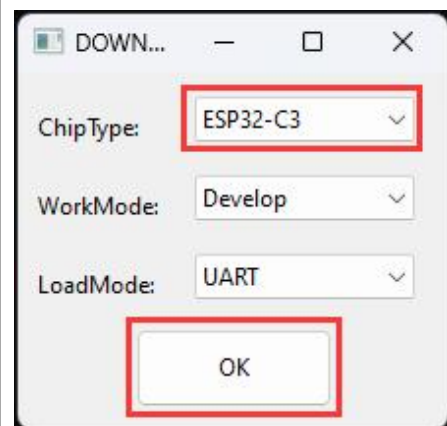


2.2.1 eArm ファームウェアの書き込み

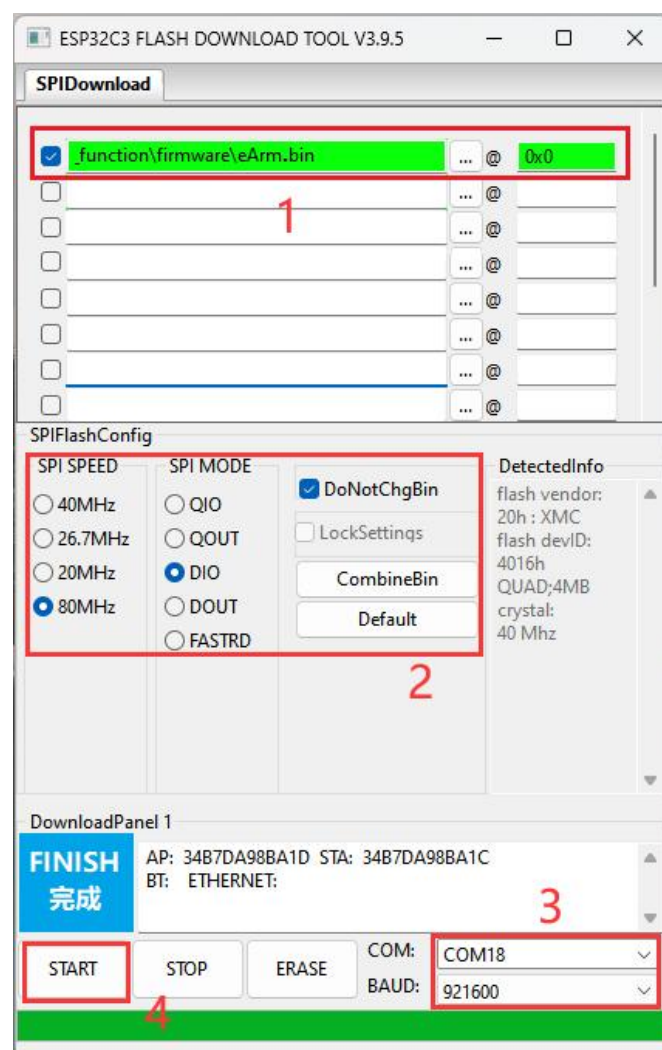
USBケーブルを使用してeArmをPCに接続します：



書き込みツールを起動します：



書き込み対象のファームウェアを選択し、書き込み先アドレスを正しく入力します。



CH340 ドライバーがインストールされており、バッテリーが正しく接続され、eArmの電源スイッチがオンになっている必要があります。そうでない場合、COMポートは検出されません。

1：書き込みオプションの「」にチェックを入れ、「」をクリックし、本チュートリアルで提供されている「**eArm.bin**」ファームウェアファイルを選択します。その後、「@」の後に書き込みアドレス（**0x0**）を正しく入力してください：

ファイル名	書き込みアドレス
eArm.bin	0x0

2：SPI SPEED で **80MHz**、SPI MODE で **DIO** を選択し、 **DotNotChgBin** にチェックを入れます。

3：制御ボード用のシリアルポートを選択します。ここでは COM18 です（お使いのコンピュータでこのデバイスに実際に割り当てられているポートに基づいて選択してください）。ボーレートを **921600** に設定します。書き込みに失敗した場合は、ボーレートを **115200** などに下げてください。

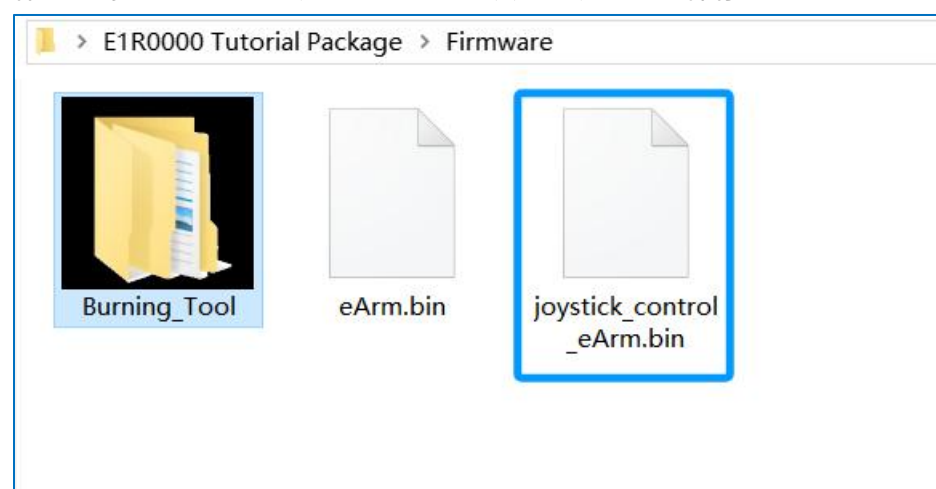
4：クリックしてファームウェアの書き込みを開始します。

書き込みが完了したら、「**組立・取扱説明書**」の手順に従ってロボットアームを操作できます！

注記：ファームウェアの書き込みが完了したら、RST（リセット）ボタンを押すか、USB ケーブルを抜いてから、再度電源を入れてください。

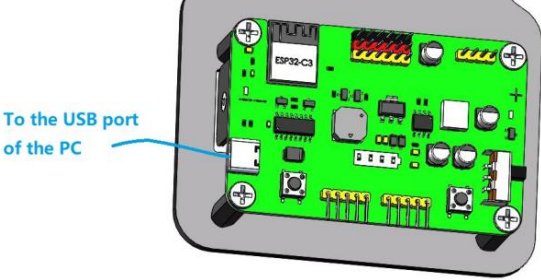
2.3 ジョイスティック制御ファームウェア（記録機能付き）

このファームウェアは、セクション 4.8.4 のコードと同じ機能を備えており、20 個の動作を記録できます。ファームウェアは次のファイルに保存されています：

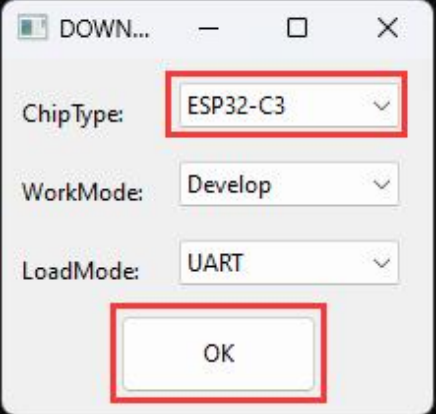


2.3.1 ジョイスティック制御ファームウェアの書き込み

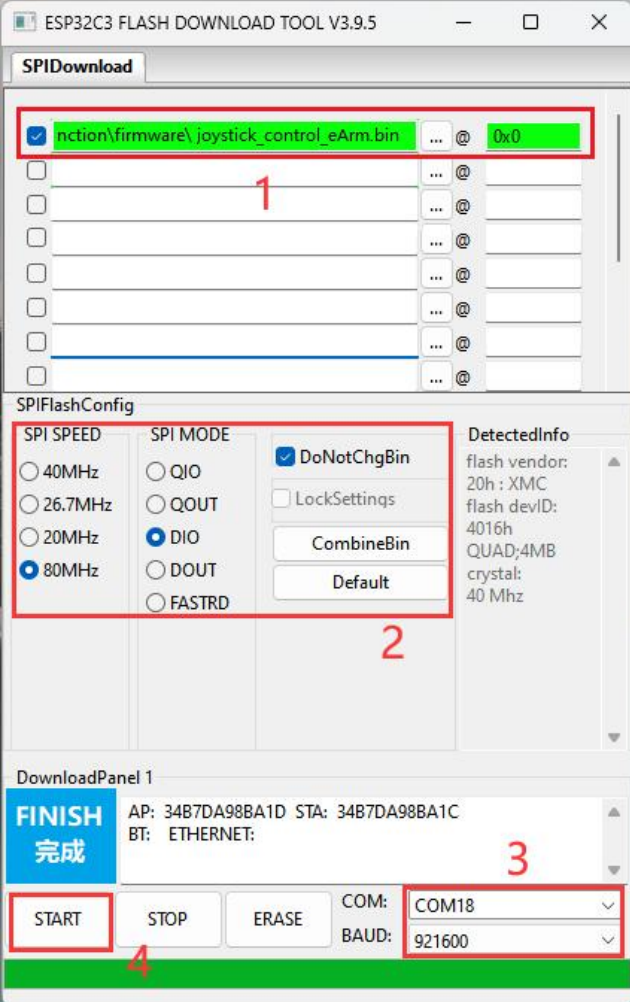
USB ケーブルを使用して eArm を PC に接続します：



書き込みツールを起動します：



書き込み対象のファームウェアを選択し、書き込み先アドレスを正しく入力します。



CH340 ドライバがインストールされていること、バッテリーが正しく接続されていること、および eArm の電源スイッチがオンになっている必要があります。これらを満たしていない場合、COM ポ

1: 書き込みオプションの 1 つ「」を選択し、「」をクリックして、チュートリアルで提供されている「**joystick_control_eArm.bin**」ファームウェアファイルを選択します。次に、「@」の後に書き込みアドレス (**0x0**) を正しく入力します。

ファイル名	書き込みアドレス
joystick_control_eArm.bin	0x0

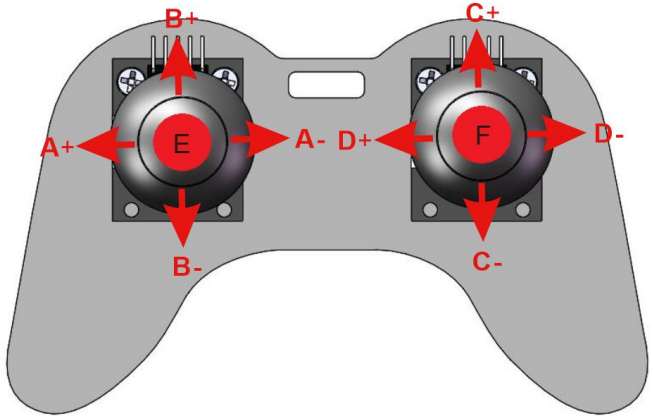
2：SPI SPEED で **80MHz**、SPI MODE で **DIO** を選択し、「」の「**DotNotChgBin**」にチェックを入れます。

3：コントロールボード用のシリアルポートを選択します。ここでは COM18 です（お使いのコンピュータでこのデバイスに実際に割り当てられているポートに基づいて選択してください）。ボーレートを **921600** に設定します。書き込みに失敗した場合は、ボーレートを **115200** などに下げてください。

4：クリックしてファームウェアの書き込みを開始します。

注記：ファームウェアの書き込みが完了したら、RST（リセット）ボタンを押すか、USB ケーブルを抜いてから、再度電源を入れてください。

ジョイスティック制御 - ロボットアーム操作ガイド：

	
A+：Aservo を制御し、ロボットアームが左に回転します。	A-：A サーボを制御し、ロボットアームが右に回転します。
B+：B サーボを制御し、ロボットアームのメインアームが前方に振れます。	B-：B サーボを制御し、ロボットアームのメインアームが後方に振れます。
C+：C サーボを制御し、ロボットアームの細かいアームが前方に振れる。	C-：C サーボを制御し、ロボットアームの細かいアームを後方に振る。
D+：D サーボを制御し、ロボットアームのグリッパーが開く（緩む）。	D-：D サーボを制御し、ロボットアームのグリッパーを閉じ（締め付け）ます。
E：動作の記録 - E ボタンを 1 回押すと 1 つの動作を記録します（最大 20 個の動作を連続して記録可能）。押すたびに短いピープ音が鳴ります。記録容量の上限に達すると、ブザーから長いピープ音が 1 回鳴ります。	F：再生 - 動作シーケンスの記録が完了したら、F ボタンを 1 回押すと、記録された動作を連続で繰り返し再生（ループ再生）します。ブザーが 1 回鳴り、機械臂が記録された動作をループで繰り返します。 ループを終了するには、F ボタンを長押ししてください。ブザーが長く 1 回鳴り、再生が停止したことをお知らせします。

3

よくある問題と解決策

Thonny IDE を使った ESP32-C3 プログラミングで発生するよくある問題と解決策
(Windows 版) - 初心者向け QA

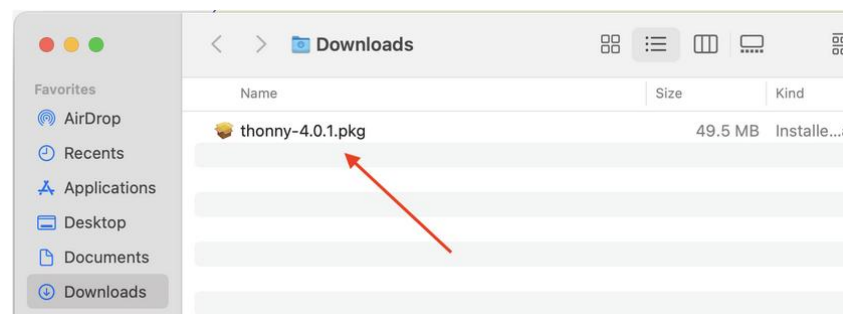
この QA は、Windows で Thonny IDE を使用して ESP32-C3 をプログラミングする際に初心者が遭遇する最も頻繁なエラー、予期しない動作、操作上の問題をまとめたものです。各セクションには、エラーの症状、根本原因、ステップバイステップの解決策、および実践的なエラーケース/スクリーンショットの説明（簡単に識別できるように）が含まれています。すべての操作は初心者になさしく、MicroPython 開発ワークフローに沿っています。

3.1 MacOS への Thonny IDE のインストール方法

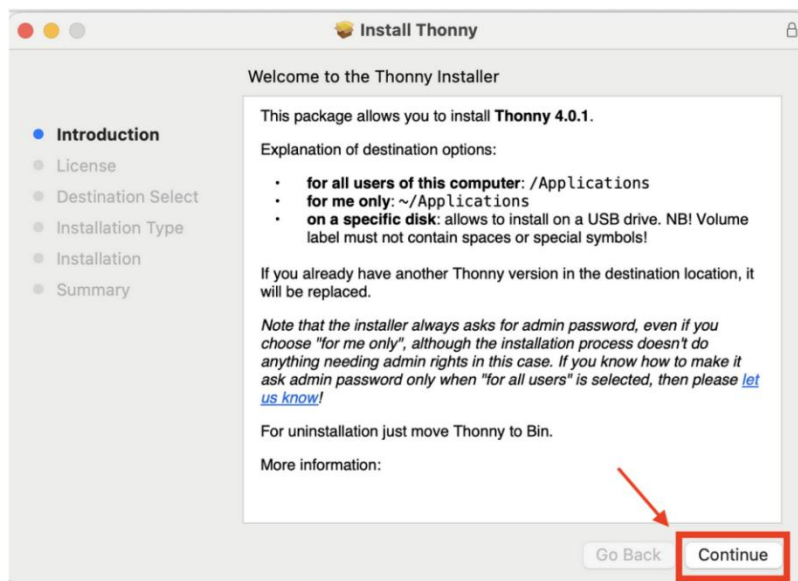
- 1) Thonny 公式ウェブサイトアクセスします：<https://thonny.org/>
- 2) [thonny-4.1.7.pkg \(42 MB\)](#) インストーラーをクリックしてダウンロードします。



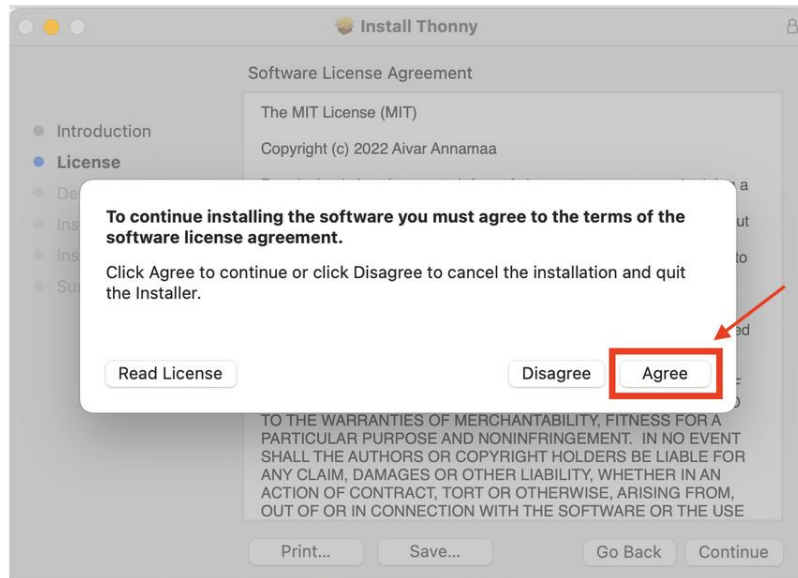
- 3) インストーラーを開いて実行し、「許可」をクリックします。



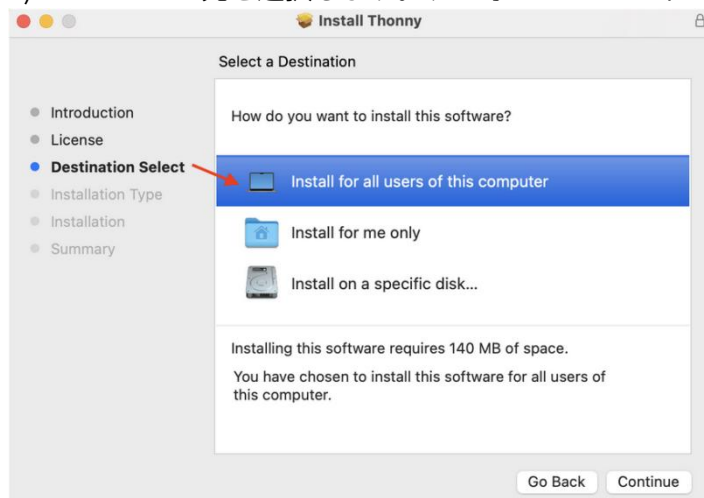
- 4) 「続ける」をクリックしてインストールを確定します。



5) 「続ける」をクリックし続け、ライセンス契約に「同意する」をクリックします

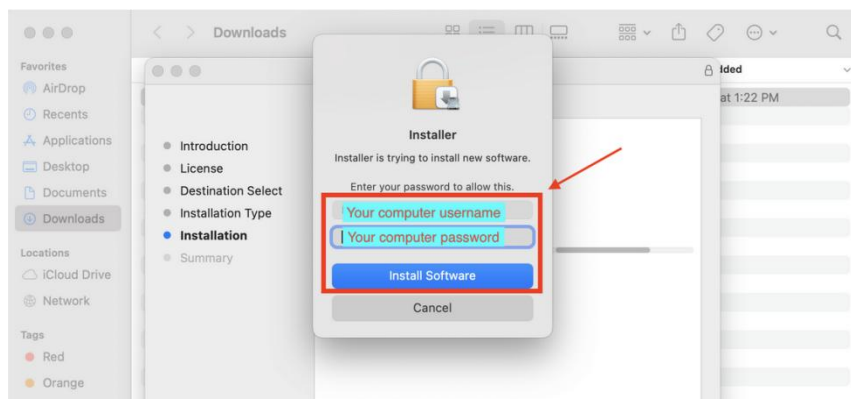


6) インストール先を選択します。デフォルトオプションを選択しても問題ありません。

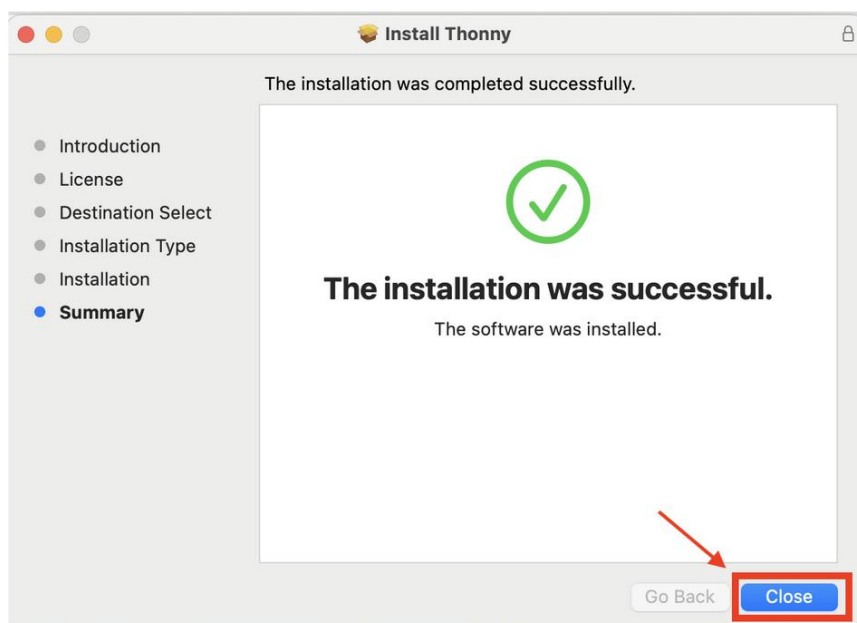


7) 「インストール」をクリックし、必要に応じて「OK」をクリックしてインストーラー

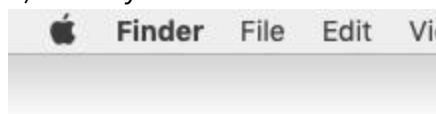
がフォルダにアクセスすることを許可します。インストールが完了するまで待ちます。



8) 「閉じる」をクリックします。



9) Thonny IDE がインストールされました。ダブルクリックして開くことができます：



3.2 ロボットアームが動作しない

組み立て中にサーボの角度を初期化するために電源スイッチをオンにしましたか？

組み立ては正しいですか？

バッテリーの電力は十分ですか？

バッテリーは仕様に従って使用されていますか？

3.3 ポート接続・デバイス認識エラー

Q1: Thonny 右下隅のポートメニューに「MicroPython (ESP32)」オプションが表示されない

エラー症状

- ＞ Thonny IDE の右下隅には「Local Python 3.x」のみが表示され、MicroPython (ESP32)オプションが存在しない
- ＞ ツール→オプション→インタープリター→MicroPython (ESP32)を開くと、「ポートまたは WebREPL」ドロップダウンメニューに「USB Serial @ COMx」ポート選択肢がない

根本原因

1. CH340 USB シリアル変換ドライバがインストールされていない (ESP32-C3 開発ボードは USB 通信にこのチップを使用し、Windows にはプリインストールされていない)
2. USB ケーブルの接続が緩い、またはデータ転送対応ケーブルではない充電専用ケーブルを使用している
3. ESP32-C3 が電源オンになっていない (USB が接続されていない、またはボードの電源スイッチがオフになっている)
4. USB ポートが損傷している (デスクトップ PC のフロントパネル USB ポートは安定性が低いことが多い)

段階的な解決方法

1. [USB シリアル変換ドライバのインストール](#) (重要なステップ) :

1.3 CH340 USB ドライバのインストールに従ってドライバをインストールする

2. USB ケーブルとポートの確認：

データ転送対応 USB ケーブルに交換する（充電専用ケーブルにはデータピンがなく、シリアルデータを送信できない）

USB ケーブルをデスクトップ PC のリアパネル USB ポート、またはノート PC の直接接続 USB ポートに挿す（USB ハブの使用を避ける）

3. Windows デバイスマネージャーでポートを確認：

Win + X キーを押し→デバイスマネージャーを選択→「ポート(COM および LPT)」セクションを展開する

ESP32-C3 が認識されている場合、「USB-SERIAL CH340 (COMx)」が表示される（x は 3、4、5 などの数字）

ポートに黄色い感嘆符が表示されている場合は、右クリック→デバイスのアンインストール→USB ケーブルを抜き差ししてドライバを再インストールする

Q2: STOP ボタンクリック時に Thonny に「could not open port 'COMx」と表示される

エラー症状

- ＞ Thonny の STOP ボタンをクリックすると、シェルに「could not open port 'COM3」のエラーが表示される
- ＞ デバイスマネージャーにはポートが表示されるが、Thonny では使用できない

根本原因

1. シリアルポートが他のソフトウェアによって占有されている（Arduino IDE、Serial Monitor、Putty、VS Code シリアルプラグインなど）
2. Thonny の前回のシリアル接続が適切に終了されていない（IDE キャッシュの問題）
3. Windows のバックグラウンドプロセスがシリアルポートを占有している

段階的な解決方法

1. すべてのシリアルポート関連ソフトウェアを閉じる：

Arduino IDE、Serial Monitor、Putty、およびシリアルポートを使用する可能性のあるその他のツールを閉じる

Windows タスクバーの非表示バックグラウンドソフトウェアを確認する（タスクバーを右クリック→タスクマネージャー→シリアルツール関連のプロセスを終了する）

2. Thonny IDE を再起動する：

Thonny を完全に閉じる（右上隅の X をクリックし、タスクマネージャーに Thonny プロセスが存在しないことを確認）

Thonny を再オープンし、ESP32-C3 のポートを再選択する

3. ESP32-C3 の USB ケーブルを抜き差しする：

USB ケーブルを抜き、2 秒待機してから再接続する—これによりシリアルポートがリセットされ、占有状態が解除される

3.4 MicroPython ファームウェア・インタープリター設定エラー

Q1: コード実行時に Thonny に「Device is busy or does not respond」と表示される

エラー症状

- Run/STOP ボタンをクリックすると、シェルに「Device is busy or does not respond」と表示される

根本原因

- ESP32-C3 に MicroPython ファームウェアがインストールされていない（新品ボードには通常ファームウェアがプリロードされていない）
- Thonny で誤ったインタープリターが選択されている（例：「MicroPython (ESP32)」の代わりに「MicroPython (ESP8266)」が選択されている）
- ボード上の MicroPython ファームウェアが破損している（フラッシュが不完全、またはファームウェアインストール中の停電による）

段階的な解決方法

1. Thonny でファームウェアをフラッシュする：

i. 4.5 節に従ってファームウェアを再フラッシュする（フラッシュ中は USB ケーブルを抜かないこと）

ii. 完了後、Close をクリックし、ESP32-C3 を再起動する（RESET ボタンを押す）

2. ファームウェアインストールの確認：

Thonny のシェルターミナルを開く（表示→シェル）

ESP32-C3 の RESET ボタンを押す—MicroPython のバージョン情報と >>> プロンプトが表示されれば成功

Q2: ファームウェアインストール中に Thonny に「Wrong board model」と表示される

エラー症状

- MicroPython ファームウェアインストール時にポップアップエラーが表示される：「Wrong board model selected. The firmware is not compatible with ESP32-C3」
- 「Verification error」または「Write error」が発生し、ファームウェアのフラッシュに失敗する

エラーケーススクリーンショットの説明

ファームウェアインストールの進捗バーが 10～50%で停止し、赤い警告が表示される：「Firmware for ESP32 (generic) is not compatible with this device. Please select ESP32-C3 as the board model」

根本原因

ファームウェアインストールページで誤ったボードモデルが選択されている（例：「ESP32-C3」の代わりに「ESP32 (generic)」が選択されている）—ESP32-C3 は RISC-V コアを使用し、汎用 ESP32 ファームウェア（Xtensa コア）は互換性がない

段階的な解決方法

4.5 節に従ってファームウェアを再フラッシュする（フラッシュ中は USB ケーブルを抜かないこと）

3.5 ランタイムエラー（ESP32 でのコード実行）

Q1: ESP32-C3 でコード実行時に「ImportError: no module named 'xxx'」が表示される

エラー症状

- Run をクリックまたは ESP32-C3 を再起動すると、REPL ターミナルに赤色のエラー「ImportError: No module named 'led'」が表示される
- カスタムモジュールをインポートできないため、メインコード（main.py など）の実行に失敗する

エラーケーススクリーンショットの説明

```
>>> import led
ImportError: No module named 'led'
>>> main.py:1: ImportError: No module named 'sensor'
```

根本原因

ケース 1：カスタムモジュール（led.py、sensor.py など）→ESP32-C3 にアップロードされていない

インポートされたモジュールファイルが Windows ローカルコンピュータにのみ存在し、ESP32-C3 のファイルシステムには存在しない

ケース 2：カスタムモジュール→ファイル名の大文字小文字が一致していない

MicroPython はファイル名の大文字小文字を区別する（例 :Led.py ≠ led.py、SENSOR.py ≠ sensor.py）

段階的な解決方法

1. カスタムモジュールの場合：モジュールファイルを ESP32-C3 にアップロードする

Thonny で不足しているモジュールファイル（led.py など）を開く

ファイル→デバイスにアップロードをクリックし、ESP32-C3 にアップロードする

Thonny のファイルパネル（MicroPython デバイス配下）でファイルが表示されていることを確認する

メインコードを再実行する、またはボードを再起動する

2. カスタムモジュールの場合：ファイル名の大文字小文字の不一致を修正する

インポートするファイル名が実際のファイル名と完全に一致することを確認する（大文字小文字、スペルともに）

例：ファイルが LedBlink.py の場合、インポートは import LedBlink と記述する必要がある（import ledblink ではない）

初心者推奨：ファイル名をすべて小文字にリネームし、大文字小文字によるエラーを回避する（ledblink.py など）

Q2: ESP32 でコード実行時に「OSError: [Errno 2] ENOENT」が表示されるエラー症状

- ・コード実行時にシェルターミナルに「OSError: [Errno 2] ENOENT」が表示される
- ・このエラーは通常、ファイル操作（ファイルを開くなど）またはモジュールインポート時に発生する

エラーケーススクリーンショットの説明

```
>>> f = open("data.txt", "r")
OSError: [Errno 2] ENOENT
>>> from led import run
OSError: [Errno 2] ENOENT
```

根本原因

ENOENT = Error NO Entity → コードがアクセスしようとしているファイル/モジュールが ESP32 のファイルシステムに存在しない（初心者にも多いエラー）

- ・ファイル操作の場合：対象ファイル（data.txt など）がボードにアップロードされていない
- ・インポートの場合：モジュールファイル（led.py など）が不足している、またはパスが間違っている

段階的な解決方法

1. ESP32-C3 にファイル/モジュールが存在することを確認する：

Thonny のファイルパネルで、ファイル（data.txt、led.py など）が MicroPython デバイス配下に存在することを確認する

不足している場合は、ファイルをボードにアップロードする

2. コード内のファイルパスを修正する：

ルートディレクトリのファイルにはファイル名のみを使用する（/flash/led.py や./led.py などの絶対/相対パスは使用しない）

正しい記述：import led / f = open("data.txt", "r")

誤った記述：import flash.led / f = open("./data.txt", "r")

3. ファイル名のタイプミスを確認する：

コード内のファイル名が実際のファイル名と一致することを確認する（余分なスペース、誤字がないこと）

例：data.txt ≠ data .txt、led.py ≠ ledd.py

Q3: Thonny (Run ボタン) ではコードが実行されるが、ESP32 再起動後に実行されない

エラー症状

- ＞ Thonny の Run ボタンをクリックすると、ESP32 のブザーが正常に鳴る
- ＞ ボードを再起動した後（USB を抜く/RESET を再押す）、ブザーが鳴らなくなる（コードが自動実行されない）

エラーケーススクリーンショットの説明

Run ボタンクリック時はシェルターミナルにコードの実行成功が表示される：`>>> # buzzer...;` 再起動後はターミナルに MicroPython のバージョンプロンプトのみが表示され、コードは実行されない

根本原因

1. コードが ESP32-C3 上に main.py として保存されていない（MicroPython は起動時に main.py を自動的に実行する）
2. main.py が Windows ローカルコンピュータに保存され、ボードにアップロードされていない
3. main.py に構文エラーが存在する（起動時の自動実行に失敗する原因となる）

段階的な解決方法

1. エントリーコードが main.py として保存され、アップロードされていることを確認する（核心的な修正）：

シングルファイルコードを使用する場合：コードを main.py として保存する（ファイル→名前を付けて保存）→ESP32 にアップロードする（ファイル→デバイスにアップロード）

モジュール式コードを使用する場合：すべてのカスタムモジュール（buzzer.py など）をアップロード→最小限の main.py を作成する（`import buzzer; buzzer.run()`など）→main.py をボードにアップロード

2. main.py が ESP32 のルートディレクトリに存在することを確認する：

Thonny のファイルパネルで、main.py が MicroPython デバイス配下に表示されていることを確認する（サブフォルダに存在しないこと）

不足している場合は、main.py を再アップロードする

3. main.py の構文エラーを確認する：

Thonny でボード上の main.py ファイルを開く（ファイルパネルでダブルクリック）

赤色の下線を確認する（Thonny の構文チェック）→エラーを修正する（コロン:の欠落、インデントエラー、ピン番号の誤りなど）

修正後の main.py を再アップロードし、ボードを再起動する

4. 自動実行のテスト：

正しい main.py をアップロードした後、ESP32 を再起動→Thonny の Run ボタンをクリックすることなく、コード（ブザー）が自動的に実行される

3.6 ファイルシステム・デバイス表示の異常

Q1: Thonny のファイルパネルに「MicroPython Device」が表示されない？ エラーケーススクリーンショットの説明

Thonny のファイルパネルに MicroPython デバイスのエントリは表示されるが、右ペインに赤色のテキストが表示される：「Could not list files on device. Check connection and try again.」

根本原因

1. コンピュータの USB ポートをコントロールボードに接続するたびにこの現象が発生するのは、Thonny に ESP32 デバイス接続の自動検出機能がないため
2. シリアルポート通信が不安定である（USB ケーブルの接続が緩い、不良な USB ポートを使用している）

段階的な解決方法

1. Thonny に ESP32 デバイスを認識させる：

STOP ボタンをクリックする、または Thonny 右下隅で「MicroPython (ESP32)」
▪ USB Serial @ COMx」を再選択する

2. シリアル接続を安定させる：

USB データケーブルを交換→デスクトップ PC のリアパネル USB ポートに挿す（ハブの使用を避ける）

USB ケーブルの接続が緩くないことを確認する（ボードの LED が断続的に点滅しないこと）

3.7 シリアルポートの占有・接続切断の問題

Q1: ESP32-C3 が Thonny からランダムに切断される（COM ポートが消失する）

エラーケーススクリーンショットの説明

Thonny 右下隅が突然「MicroPython on COM3」から「Local Python 3.x」に切り替わる；デバイスマネージャーのポートセクションに USB-SERIAL CH340 ポートが表示されなくなる

根本原因

3. USB 電源供給が不安定である（ノート PC/USB ハブが ESP32 に十分な電力を供給できていない）
4. ESP32 ボード上の USB シリアル変換チップが損傷している（ハードウェアの問題）
5. Windows の USB ドライバが競合している（古い CH340 ドライバ）

段階的な解決方法

1. USB 電源供給を改善する（最も一般的な修正方法）：

ノート PC の場合：USB ケーブルを給電式 USB ハブに挿す（またはノート PC を AC アダプタに接続する）

デスクトップ PC の場合：常にリアパネルの USB ポートを使用する（フロントパネルより安定した電力供給が得られる）

非給電式 USB ハブの使用を避ける（ESP32-C3 に十分な電流を供給できない）

4

お問い合わせ

上記で解決策が見つからない場合は、サポートチームにお問い合わせください。

迅速なサポートのため、以下の情報をご用意ください：

- ✧ ご注文番号
- ✧ 製品モデル（例：Robot Arm Kit E1R0000）とソフトウェアバージョン
- ✧ 問題やご質問の詳細な説明
- ✧ 既にお試しになった手順
- ✧ 関連するエラーメッセージのスクリーンショット、写真、コードスニペット

その他のお問い合わせ？

- ✧ 以下のような場合も、お気軽にお問い合わせください：
- ✧ チュートリアルの誤りとフィードバック：ドキュメントの改善にご協力ください。
- ✧ 製品のアイデアと提案：素晴らしいアイデアをお聞かせください。
- ✧ パートナーシップとコラボレーション：一緒に仕事をしてみませんか？
- ✧ 割引とプロモーション：教育機関向けや大量購入の価格についてお問い合わせください。
- ✧ その他：技術的な質問以外のことでもお気軽にどうぞ。



support@siyeenove.com



<https://siyeenove.com>